



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Kierunek studiów: Informatyka

Rodzaj studiów: Zaoczne

Praca dyplomowa

Temat pracy: System do zarządzania infrastrukturą sprzętową

Temat w języku angielskim: Hardware infrastructure management system

Promotor pracy: mgr inż. Paweł Pisarski

Wykonawcy:

Nazwisko, imię	Nr albumu
Kopczyński, Adrian	s26990
Kośmider, Patryk	s26985
Orlikowski, Ziemowit	s27081
Stankowski, Aleksander	s27549

Streszczenie: Praca dotyczy systemu zarządzania infrastrukturą sprzętową przeznaczonego do zastosowania w serwerowniach oraz laboratoriach sprzętowych. W odpowiedzi na rosnącą złożoność środowisk sprzętowych przeprowadzono analizę dostępnych rozwiązań rynkowych oraz badanie ankietowe wśród potencjalnych użytkowników systemu

Celem pracy było zaprojektowanie i implementacja systemu wspomagającego zarządzanie zasobami sprzętowymi oraz przestrzenią, z uwzględnieniem integracji z narzędziami monitorującymi. Praca obejmuje wszystkie etapy cyklu życia oprogramowania - od fazy koncepcyjnej, poprzez analizę wymagań, projekt i implementację, aż po przygotowanie produktu końcowego.

Rezultatem pracy jest funkcjonalny system umożliwiający ewidencję sprzętu, zarządzanie jego lokalizacją oraz integrację z systemem Prometheus, który może stanowić podstawę do dalszego rozwoju i wdrożenia w środowisku produkcyjnym.

Słowa kluczowe: infrastruktura sprzętowa, laboratorium sprzętowe, zasoby sprzętowe, narzędzia monitorujące, ewidencja sprzętu

Gdańsk, Lipiec, 2026



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: System do zarządzania infrastrukturą sprzętową	Akronim: Labbyn Data ustalenia tematu 2025-10-26
Promotor: mgr inż. Paweł Pisarski	Konsultanci: 1. — brak —
Cele projektu: Stworzenie systemu zarządzania infrastrukturą sprzętową który umożliwia łatwy dostęp do statystyk, lokalizacji i właściwości sprzętu	
Rezultaty projektu: W pełni funkcjonalny system zarządzania infrastrukturą sprzętową w laboratorium sprzętowym lub serwerowni	
Miary sukcesu: 1. Implementacja wszystkich wymagań oznaczonych jako “must” 2. Ukończenie prac nad systemem przed datą obrony pracy	
Ograniczenia: 1. Czas realizacji projektu: 8 miesięcy 2. Ograniczone zasoby sprzętowe: brak możliwości testowania systemu w warunkach rzeczywistych. 3. Ograniczone środki finansowe: istotne w przypadku konieczności skorzystania z usług zewnętrznych, takich jak hosting serwera do przechowywania danych lub synchronizacji. Wymaga to uwzględnienia potencjalnych dodatkowych kosztów. 4. Doświadczenie zespołu: Projekt realizowany przez grupę studencką. 5. Brak dostępu do informacji zwrotnych odnośnie systemu: ograniczona współpraca z podmiotami wykorzystującymi infrastrukturę sprzętową.	

Wykonawcy	Numer al- bumu	Specjalizacja	Tryb studiów
Adrian Kopczyński	s26990	Cyberbezpieczeństwo	Niestacjonarny
Aleksander Stankowski	s27549	Cyberbezpieczeństwo	Niestacjonarny
Patryk Kośmider	s26985	Sztuczna Inteligencja	Niestacjonarny
Ziemowit Orlikowski	s27081	Sztuczna Inteligencja	Niestacjonarny

Data ukończenia projektu: 1 maja 2026	Recenzent: BRAK
---	---------------------------

Spis treści

1	Informacje wstępne	5
1.1	Opis problemu	5
1.2	Przegląd istniejących rozwiązań	6
1.3	Proponowane rozwiązanie	7
2	Słownik pojęć	8
3	Analiza	10
3.1	Identyfikacja i eksploracja pomysłów	10
3.2	Badanie ankietowe	10
3.3	Sprawozdanie z ankiety	10
3.3.1	Cel ankiety	10
3.3.2	Metodologia ankiety	11
3.3.3	Charakterystyka grupy badawczej	11
3.3.4	Zakres i struktura ankiety	12
3.3.5	Analiza wyników ankiety	13
3.3.6	Wnioski końcowe	19
3.4	Diagramy	20
3.4.1	Główni Aktorzy	20
3.4.2	Diagramy przypadków użycia	21
3.4.3	Diagramy sekwencji	23
4	Inżynieria wymagań	25
4.1	Specyfikacja wymagań	25
4.2	Udziałowcy	26
4.3	Wymagania ogólne i dziedzinowe	27
4.4	Wymagania Funkcjonalne	29
4.4.1	Zakładka 'Labs'	29
4.4.2	Panel Urządzenia	33
4.4.3	Zakładka 'Inventory'	44
4.4.4	Zakładka 'Dashboard'	48
4.4.5	Search Bar (Pasek wyszukiwania)	49
4.4.6	Zakładka 'History'	50

4.4.7	Zakładka 'Users'	53
4.4.8	Zakładka 'Groups'	55
4.4.9	Panel Administratora	58
4.4.10	Import & Export	63
4.4.11	Zakładka 'Documentation'	64
4.4.12	Etykiety	66
4.5	Interfejs z otoczeniem	68
4.6	Wymagania pozafunkcjonalne	70
4.7	Wymagania na środowisko docelowe	72
5	Architektura systemu Backend	73
5.1	Stack technologiczny	73
5.2	Architektura i wzorce projektowe	73
5.2.1	Warstwa interfejsu	73
5.2.2	Warstwa serwisowa	74
5.2.3	Warstwa egzekutorów	75
5.2.4	Warstwa dostępu do danych	77
5.3	Wzorce projektowe	78
5.3.1	Wstrzykiwanie zależności	79
5.3.2	Singleton	79
5.3.3	Data Transfer Object	80
5.3.4	Observer	82
5.4	Konfiguracja i środowisko	83
5.5	Modele danych	87
5.6	Autentykacja	90
6	Przebieg sprintów	94
6.1	Sprint 0	94
6.2	Sprint 1	95
6.3	Sprint 2	98
6.4	Sprint 3	99
6.5	Sprint 4	99
6.6	Sprint 5	99
6.7	Sprint 6	99
6.8	Sprint 7	99
6.9	Sprint 8	99
6.10	Sprint 9	99
6.11	Sprint 10	99
6.12	Sprint 11	99
7	Testy	100

8 Zarządzanie projektem	101
8.1 Planowanie pracy zespołu i środki komunikacji	101
8.2 Narzędzia wspomagające	101
8.3 Harmonogram Prac	101
9 Nakład Prac	103
9.1 Całościowy nakład prac	103
9.2 Indywidualny nakład prac	103
9.2.1 Adrian Kopczyński	103
9.2.2 Aleksander Stankowski	103
9.2.3 Patryk Kośmider	103
9.2.4 Ziemowit Orlikowski	103
10 Wnioski końcowe	104
11 Załączniki	105
Spis rysunków	106
Bibliografia	108

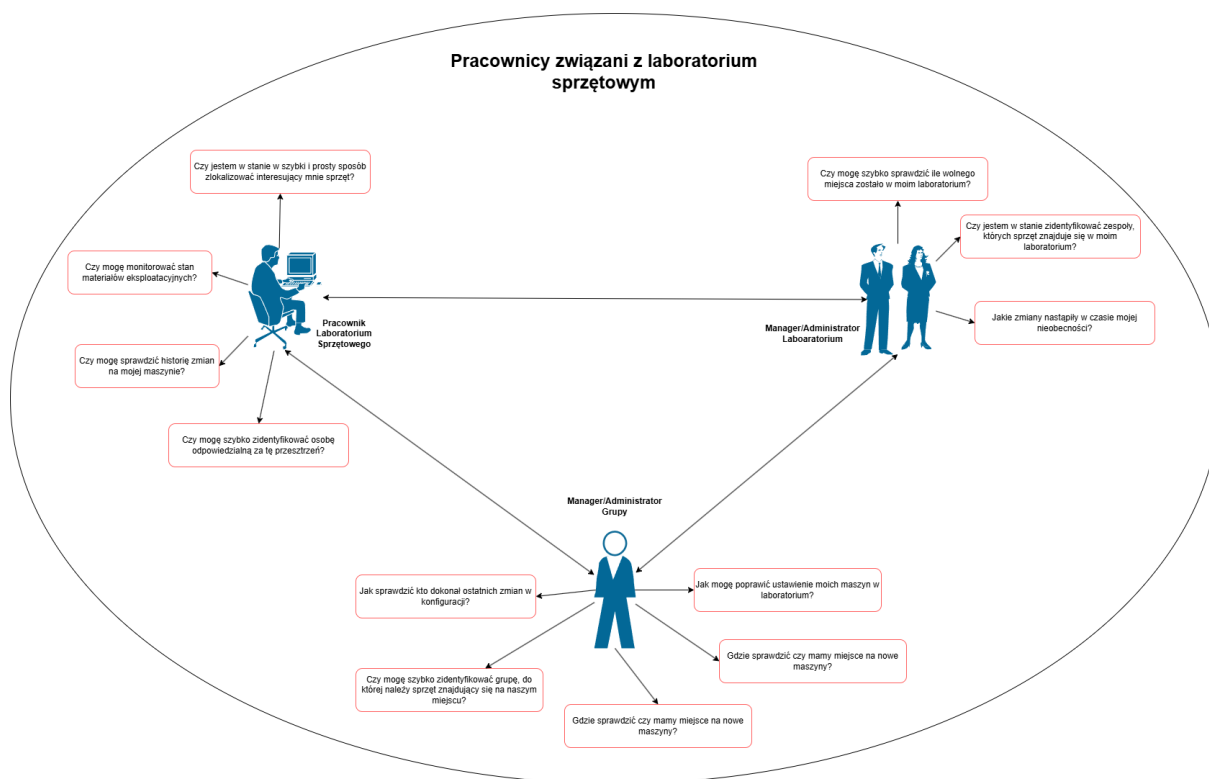
1 Informacje wstępne

1.1 Opis problemu

Motywacją powstania projektu były zawodowe doświadczenia członków zespołu projektowego. Podczas pracy ze sprzętem komputerowym, na różnorodnych stanowiskach, elementem łączącym wszystkie te stanowiska był brak odpowiedniego oprogramowania do efektywnego zarządzania oraz monitorowania infrastruktury sprzętowej w jednym, scentralizowanym systemie łączącym funkcje ewidencji sprzętu z telemetrią.

W literaturze podkreśla się, że organizacje napotykają istotne bariery we wdrażaniu podobnych systemów. W swoich badaniach Damjan Maletic, Marta Grabowska oraz Matja Maletic wskazują, że głównymi czynnikami tego stanu rzeczy są bariery finansowe i organizacyjne, co przekłada się na niską adopcję narzędzi wspierających ewidencję i monitorowanie zasobów. [1] W kontekście zarządzania infrastrukturą informatyczną wskazane bariery mogą prowadzić do rezygnacji z wdrażania dedykowanych systemów ewidencji sprzętu lub ograniczenia ich funkcjonalności do minimum.

Z tego względu zespół projektowy podjął się stworzenia systemu open-source przyjaznego użytkownikowi zarówno w obsłudze, utrzymaniu jak i wdrożeniu co jest bezpośrednią odpowiedzią na problemy lub niechęć organizacji do stosowania tego typu systemów.



Rysunek 1.1: Rich Picture

1.2 Przegląd istniejących rozwiązań

Na rynku dostępny jest szereg gotowych rozwiązań problemu postawionego w tej pracy inżynierskiej. Większość z nich jest płatna oraz opiera się na przetrzymywaniu danych w chmurze, a nie lokalnie co dla wielu organizacji jest kolejnym potencjalnym ryzykiem, na które nie mogą sobie pozwolić. Przykładowe rozwiązania:

- **RackTables** <https://www.racktables.org>
- **Nautobot** <https://networktocode.com/nautobot/>
- **Device42** <https://www.device42.com>

Podobnym projektem jest produkt firmy NetBox Labs, Inc. Jest on niewątpliwie rozwiązaniem konkurencyjnym pod względem funkcjonalności jakie oferuje. Umożliwia ono szereg funkcji od zarządzania platformami, połączeniami sieciowymi po rozproszony system kontroli wersji z możliwością działania w pełni w chmurze. Jest to system wiele bardziej zaawansowany od zaprojektowanego w ramach tej pracy dyplomowej, jednak jest on systemem płatnym, posiadającym bardzo okrojone darmowe wersje, a także wymagającym skomplikowanego planowania jak i wdrożenia, czego potwierdzenie można zauważyć na forach dyskusyjnych:

- https://www.reddit.com/r/Netbox/comments/1rjrsga/netbox_install_newbie/

- https://www.reddit.com/r/Netbox/comments/1g3b619/netbox_installation_problem/
- https://www.reddit.com/r/Netbox/comments/1ondzhn/stuck_in_last_step_of_installation/
- https://www.reddit.com/r/Netbox/comments/1iqdmxw/install_issues_with_django_and_fix/

W przeciwieństwie do naszego rozwiązania nie posiada on również mapy przestrzennej laboratorium.

1.3 Proponowane rozwiązanie

Celem realizacji projektu, jest stworzenie systemu zarządzania i monitorowania inwentarzem technicznym, który będzie w stanie sprostać podstawowym potrzebom w pracy ze sprzętem komputerowym. Cel pracy został ograniczony do kluczowych funkcjonalności systemu ze względu na ograniczenia projektowe. Oprogramowanie celowane jest do organizacji bez podobnego rozwiązania zatem wdrożenie systemu musi być proste oraz pozwolić w łatwy sposób przenieść aktualnie kompletowane dane. Cechą kluczową systemu jest mapa przestrzenna laboratorium, która oprócz zwiększenia efektywności prac, ułatwia proces onboarding'u pozwalając zapoznać się z zasobami organizacji w sposób zorganizowany z ułatwionym i zcentralizowanym dostępem do informacji.

2 Słownik pojęć

Ansible narzędzie służące do automatycznego konfigurowania systemów. 69, 73, 75

API Zestaw reguł pozwalający na komunikację pomiędzy poszczególnymi częściami aplikacji. 73, 85

atrybuty obliczalne metoda klasy, która może być wywoływana jako zwykła zmienna. 88

Backend “Serce” systemu odpowiedzialne za logikę, obliczenia, bezpieczeństwo i dostarczanie danych które mają zostać odebrane oraz przekazane użytkownikowi. 73

CORS (z ang. Cross-Origin Resource Sharing), mechanizm bezpieczeństwa przeglądarek, który pozwala na określenie konkretnych reguł na podstawie których można odbierać żądania użytkownika. 85

Epik Duży zbiór zadań lub funkcja systemu, która jest zbyt obszerna, aby zrealizować ją w jednym sprincie. 102

FastAPI Framework języka Python, służący do budowania zaawansowanych interfejsów API. 73, 79, 90, 91

Grafana platforma służąca do wizualizacji, monitorowania i analizy danych w czasie rzeczywistym.. 73

Labbyn System do zarządzania infrastrukturą sprzętową. 1, 10, 20, 26, 69, 72, 79, 80, 82, 83, 87, 91

laboratorium Pomieszczenie przeznaczone do przechowywania sprzętu komputerowego. 1, 7, 27–30, 33

meeting minutes Zwięzła notatka służbowa, która dokumentuje najważniejsze ustalenia, decyzje oraz przypisane zadania z przeprowadzonych rozmów. 101

onboarding Ustrukturyzowany proces wdrażania nowego pracownika do firmy. 7

open-source Rodzaj oprogramowania komputerowego, którego kod źródłowy jest publicznie dostępny. 5

OpenAPI ujednolicony standard interfejsów REST API. 73

ORM (ang. Object-Relational Mapping) technika mapowania obiektów w kodzie na tabele w bazie. 73, 77

PDU Power Distribution Unit, urządzenie wyposażone w wiele gniazd elektrycznych. 16, 37, 68

Playbook Plik w formacie YAML, który definiuje serię zautomatyzowanych zadań do wykonania. 69

POC (ang. Proof of Concept) prototyp potwierdzający, że dany pomysł jest możliwy do wykonania i sensowny z biznesowego punktu widzenia. 94, 96

PostgreSQL obiektowo-relacyjny system zarządzania bazą danych. 73

Prometheus System telemetry i monitorowania urządzeń. 1, 69, 73, 75, 84

Pull Request Mechanizm który powiadamia o zakończeniu prac nad funkcją i prosi o przejrzenie oraz scalenie zmian do docelowej gałęzi repozytorium. 94

Pydantic biblioteka Python, odpowiedzialna za walidację danych wejściowych do backendu. 73, 80

race condition błąd logiczny, w którym wynik operacji zależy od nieprzewidywalnej kolejności wykonania współbieżnych procesów. 73

rack Szafa rack, szafa teleinformatyczna, szafa serwerowa, szafa krosownicza. 35

Redis baza danych przechowująca dane w pamięci RAM. 73, 80

Router Komponent FastAPI, który pozwala na tworzenie interfejsów API z mniejszych, niezależnych modułów. 86

sprint Krótka, ograniczona czasowo iteracja, podczas której zespół pracuje nad stworzeniem gotowego, działającego przyrostu produktu. 3, 94, 95, 98, 99, 101

SQLAlchemy Biblioteka języka Python umożliwiająca komunikację z bazami danych oraz mapowanie obiektowo-relacyjne. 73, 87, 98

3 Analiza

3.1 Identyfikacja i eksploracja pomysłów

Celem etapu było wygenerowanie możliwie szerokiego zbioru pomysłów dotyczących funkcjonalności i architektury projektowanego systemu, bazując na doświadczeniu członków zespołu oraz wstępnych obserwacjach problemu. Zastosowano klasyczną technikę burzy mózgów gdzie każdy z członków zespołu projektowego był zarówno pomysłodawcą jak i oceniającym. Pełniła ona nie tylko funkcję generowania pomysłów, lecz także miała istotny wymiar integracyjny, zadziałała jako mechanizm inicjujący proces formowania zespołu: stworzyła bezpieczną przestrzeń wymiany opinii, zredukowała bariery komunikacyjne oraz umożliwiła członkom zespołu na wypracowanie wspólnego sposobu dyskusji nad rozwiązaniami. Pomysły rejestrowano w ustrukturyzowanej formie (lista funkcjonalności, możliwe komponenty systemu, scenariusze użycia). W wyniku sesji uzyskano wstępną koncepcję systemu, która była podstawą do dalszych działań.

3.2 Badanie ankietowe

W ramach udoskonalania conceptów oraz poszukiwania obszarów, które nie zostały w pełni pokryte przez dotychczasową analizę, przeprowadzone zostało badanie ankietowe. Grupą docelową ankiety były osoby mające styczność z laboratoriami sprzętowymi w swojej pracy.

3.3 Sprawozdanie z ankiety

3.3.1 Cel ankiety

Ankieta miała na celu rozpoznanie oraz doprecyzowanie wymagań dla systemu Labbyn, służącego do zarządzania platformami w serwerowni oraz laboratorium. Celem badania było zebranie opinii wśród użytkowników końcowych oraz administratorów, aby jak najlepiej dostosować funkcjonalności systemu do rzeczywistych potrzeb. Ankieta została przeprowadzona przez zespół pracujący nad projektem Labbyn w składzie: Patryk Kośmider, Ziemowit Orlikowski, Adrian Kopczyński, Aleksander Stankowski.

Czas trwania ankiety: 23.10.2025 - 29.10.2025

Łącznie odpowiedziało 14 osób

3.3.2 Metodologia ankiety

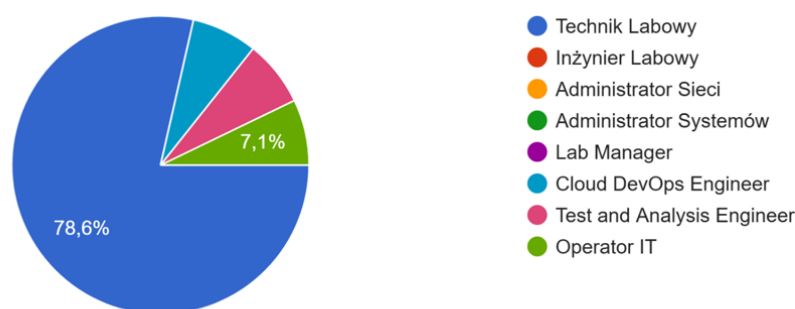
Ankieta została przeprowadzona w formie online, za pomocą formularza Google. Badanie było anonimowe, by pozwolić użytkownikom na swobodę w wyrażaniu swoich opinii. Zaproszone do ankiety zostały osoby pracujące w środowisku IT - szczególnie technicy oraz użytkownicy laboratoriów sprzętowych.

3.3.3 Charakterystyka grupy badawczej

W ankiecie wzięli udział pracownicy działów technicznych, którzy posiadają styczność ze sprzętem oraz zajmują się jego zarządzaniem. Wśród badanych znalazło się:

1. Jaką pełnisz rolę w zespole?

14 odpowiedzi

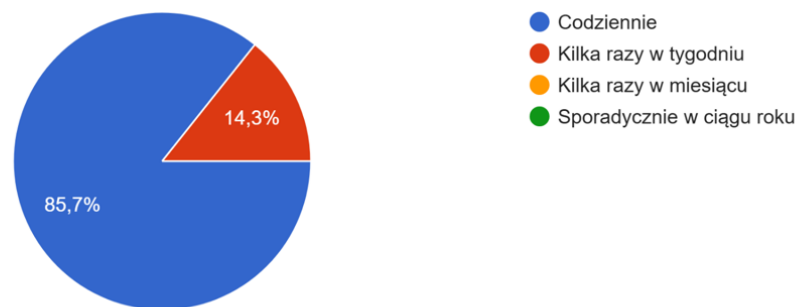


Rysunek 3.1: Wykres kołowy uzyskanych odpowiedzi na pytanie 1.

- 11 osób na pozycji Technik Labowy (78,6%)
- 1 osoba na pozycji Operator IT (7,1%)
- 1 osoba na pozycji Test and Analysis Engineer (7,1%)
- 1 osoba na pozycji Cloud Devops Engineer (7,1%)

2. Jak często masz kontakt z infrastrukturą sprzętową?

14 odpowiedzi

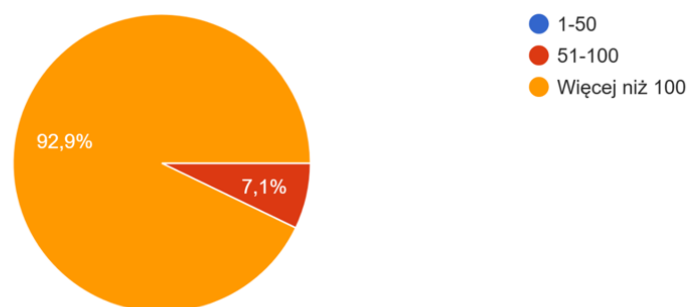


Rysunek 3.2: Wykres kołowy uzyskanych odpowiedzi na pytanie 2.

Ponad 85% ankietowanych osób, ma styczność z infrastrukturą sprzętową codziennie.

3. Jak dużo maszyn znajduje się w twoim labie?

14 odpowiedzi



Rysunek 3.3: Wykres kołowy uzyskanych odpowiedzi na pytanie 3.

Liczba maszyn, znajdujących w laboratoriach / serwerowniach wśród osób pracujących, w ponad 90% wynosi więcej niż 100. Świadczy to o złożoności i dużej skali infrastruktury, która wymaga centralnego zarządzania.

3.3.4 Zakres i struktura ankiety

Ankieta składała się z 11 pytań, podzielonych na dwie części.

- Część I - charakterystyka respondentów (rola w zespole, kontakt z infrastrukturą, liczba maszyn w laboratoriach)
- Część II - potrzeby, obecne w pracy frustracje, wymagania wobec systemu.

Większość pytań miała charakter zamknięty (jednokrotny lub wielokrotny wybór). Ostatnie pytanie było w formie otwartej, pozwalając użytkownikowi na pozostawienie dłuższych, opisowych uwag oraz sugestii. Pytania wyglądały następująco:

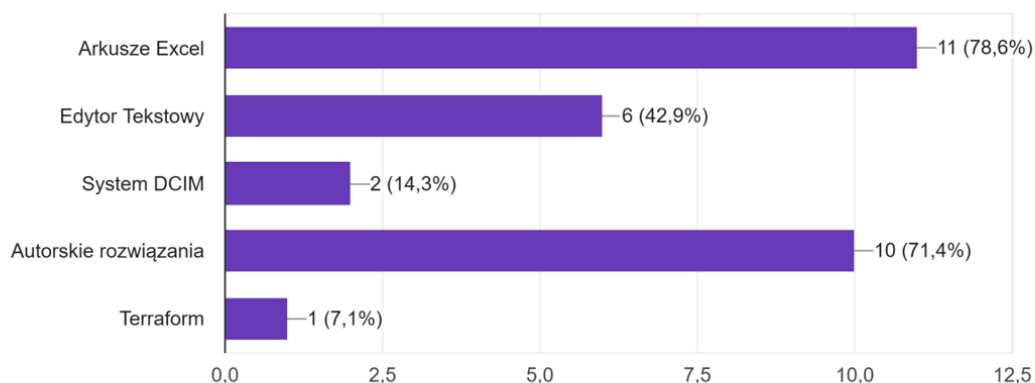
1. Jaką pełnisz rolę w zespole?
2. Jak często masz kontakt z infrastrukturą sprzętową?
3. Jak dużo maszyn znajduje się w twoim labie?
4. Z jakich narzędzi korzystasz w celu zarządzania sprzętem?
5. Co frustruje cię w codziennej pracy ze sprzętem i jego zarządzaniem?
6. Jakie funkcje byłyby dla ciebie najbardziej przydatne w takim systemie?
7. Jakie dane o konkretnym hoście chciałbyś/abyś widzieć w interfejsie?
8. Jaki informacje o sprzęcie w inwentarzu technicznym byłyby dla ciebie najbardziej pomocne?
9. W jakiej formie najchętniej korzystałbyś/abyś z takiego systemu?
10. Czy byłbyś/abyś zainteresowany przetestowaniem wersji demo systemu?
11. Czy masz jakieś dodatkowe uwagi dot. systemu?

3.3.5 Analiza wyników ankiety

Analiza dot. pytań 1-3 (kierowanych na rozpoznanie respondentów, znajduje się w podrozdziale 3.3.3 Charakterystyka grupy badawczej).

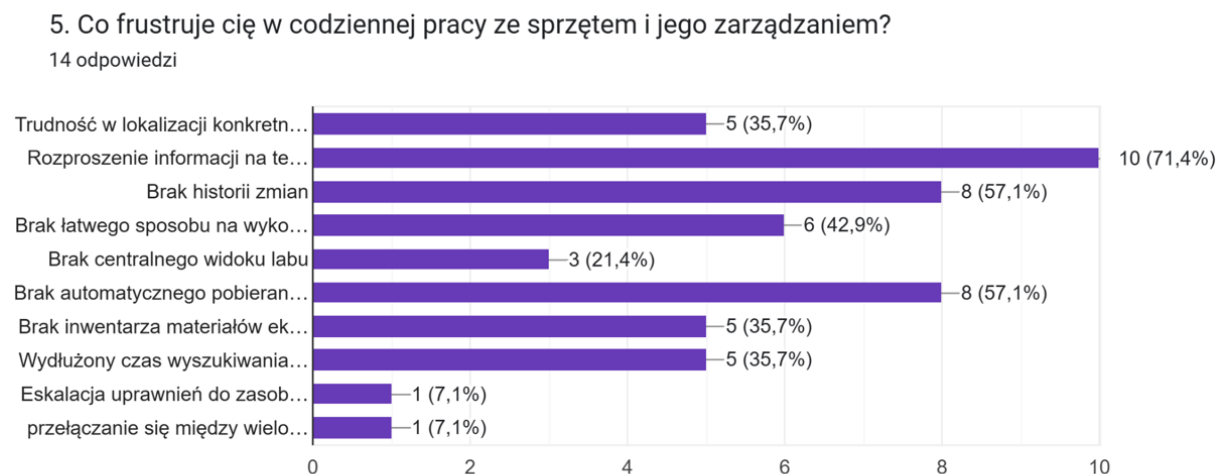
4. Z jakich narzędzi korzystasz w celu zarządzania sprzętem?

14 odpowiedzi



Rysunek 3.4: Wykres uzyskanych odpowiedzi na pytanie 4.

Najczęściej wykorzystywane narzędzia do zarządzania laboratorium, to Excel (78,6%) oraz Autorskie rozwiązania (71,4%). Bardzo mało, ponieważ zaledwie 14,3% respondentów określiło, że korzysta z gotowych, dedykowanych do tego narzędzi, zwanych systemami DCIM. Brak jednego zunifikowanego systemu prowadzi do dużego rozproszenia danych i utrudnionej pracy zespołowej.



Rysunek 3.5: Wykres uzyskanych odpowiedzi na pytanie 5.

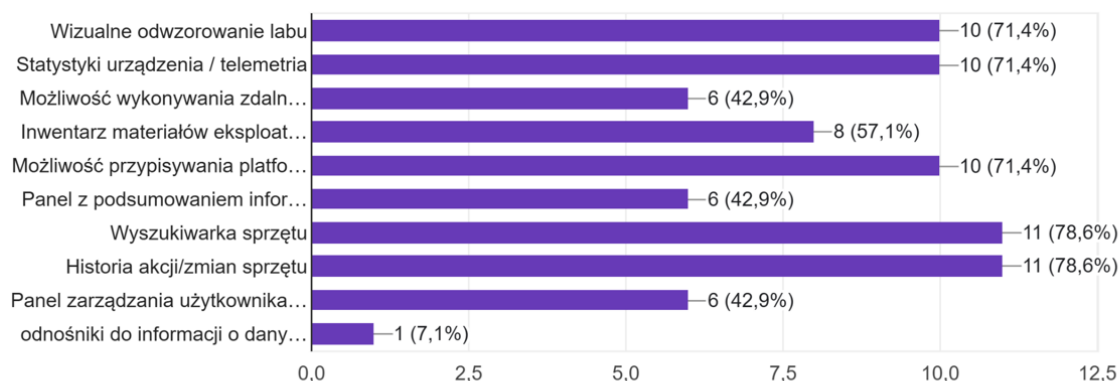
Respondenci spólnie wskazali kluczowe frustracje:

- Rozproszenie informacji na temat sprzętu w wielu źródłach (71,4%)
- Brak historii zmian (57,1%)
- Brak automatycznego pobierania danych o sprzęcie (57,1%)
- Brak inwentarza materiałów eksploatacyjnych (35,7%)
- Brak łatwego sposobu na wykonywanie akcji zdalnych (reboot, ping) (42,9%)
- Wydłużony czas wyszukiwania danych (35,7%)

Wielu techników podkreślało, że obecne metody (Excel, notatki, skrypty) nie skalują się przy dużej liczbie maszyn i powodują chaos informacyjny.

6. Jakie funkcje byłyby dla ciebie najbardziej przydatne w takim systemie?

14 odpowiedzi



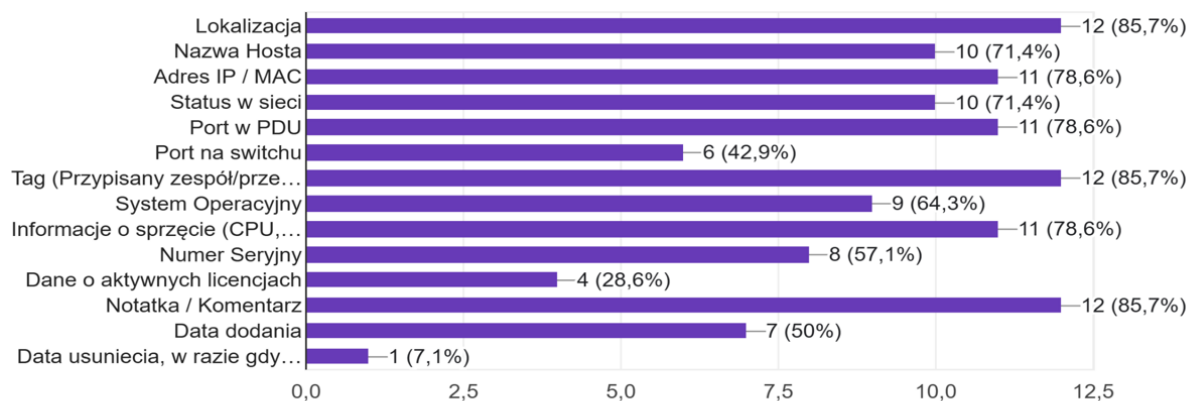
Rysunek 3.6: Wykres uzyskanych odpowiedzi na pytanie 6.

Respondenci ponownie jednomyślnie, określili jakich funkcjonalności najbardziej oczekiwali by od takowego systemu. Z odpowiedzi, wynika największe zapotrzebowanie jest na:

- Wizualne odwzorowanie laboratorium (71,4%)
- Statystyki i telemetria urządzeń (71,4%)
- Możliwość wykonywania zdalnych akcji (np. reboot, ping) (42,9%)
- Inwentarz materiałów eksploatacyjnych (57,1%)
- Możliwość przypisywania platform do konkretnych racków i półek (71,4%)
- Panel z podsumowaniem informacji o sprzęcie przypisanym do użytkownika (42,9%)
- Historia akcji i zmian sprzętu (78,6%)
- Panel zarządzania użytkownikami (role, uprawnienia) (42,9%)
- Wyszukiwarka sprzętu (78,6%)

7. Jakie dane o konkretnym gościu chciałbyś/abyś widzieć w interfejsie?

14 odpowiedzi



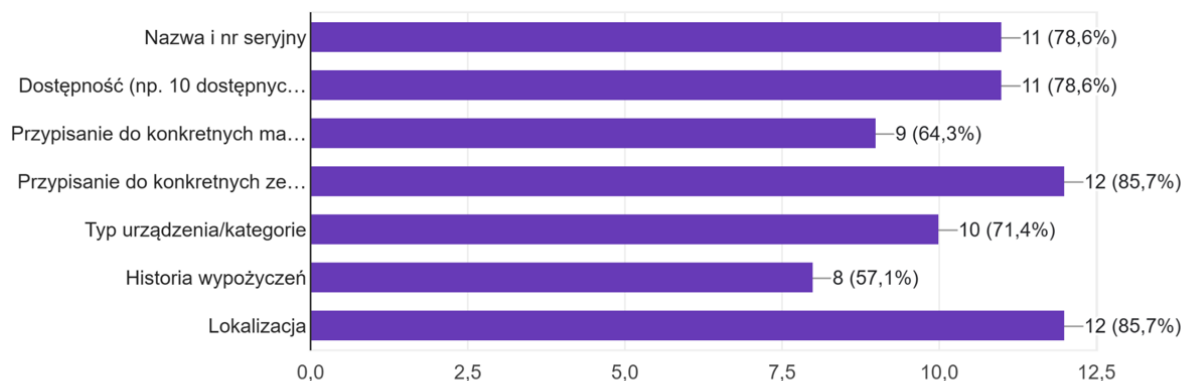
Rysunek 3.7: Wykres uzyskanych odpowiedzi na pytanie 7.

W informacjach o gościu, dostępnych pod “ręką”, użytkownicy jako najważniejsze dane wymienili:

- Lokalizacja (85,7%)
- Nazwa Hosta (71,4%)
- Adres IP oraz MAC (78,6%)
- Status w sieci (71,4%)
- Port w PDU (78,6%)
- Tag (przypisany zespół / przeznaczony zespół) (85,7%)
- System Operacyjny (64,3%)
- Informacje o sprzęcie (CPU, Dysk, RAM) (78,6%)
- Numer Seryjny (57,1%)
- Notatka/Komentarz (85,7%)
- Data dodania (50%)

8. Jaki informacje o sprzęcie w inwentarzu technicznym byłyby dla ciebie najbardziej pomocne?

14 odpowiedzi

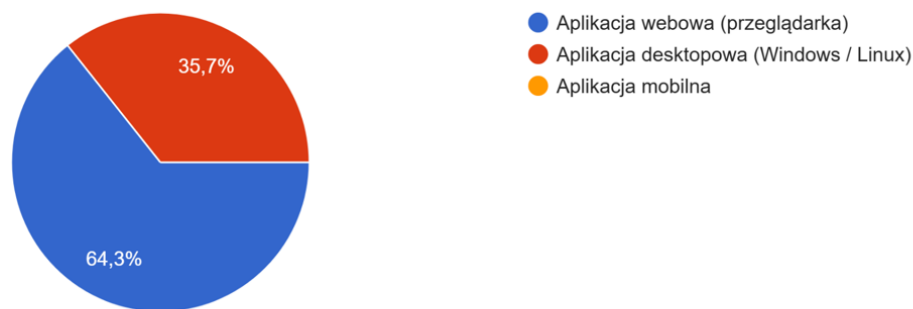


Rysunek 3.8: Wykres uzyskanych odpowiedzi na pytanie 8.

W odpowiedzi na to jakie informacje o sprzęcie potencjalni użytkownicy oczekują w funkcjonalności “inwentarz techniczny”, odpowiedzi były jednoznaczne. Wszystkie z wymienionych informacji zostały uznane za bardzo pomocne.

9. W jakiej formie najchętniej korzystałbyś/abyś z takiego systemu?

14 odpowiedzi

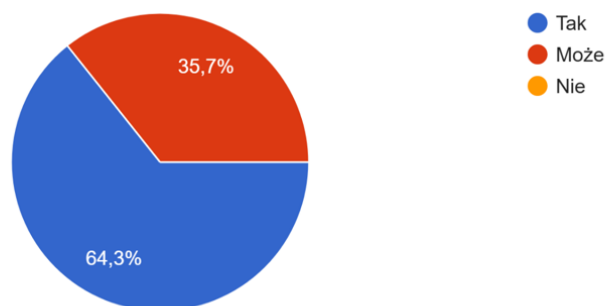


Rysunek 3.9: Wykres uzyskanych odpowiedzi na pytanie 9.

W odpowiedzi na to, w jakiej formie najchętniej korzystali z takiego systemu, najczęściej odpowiedzi uzyskała opcja Aplikacja webowa (przeglądarkowa), dobijając 64,3% głosów. Pozostała część respondentów wypowiedziała się za aplikacją desktopową. Zespół projektowy rozważa, utworzenie formy hybrydowej, jednak większy nacisk zostanie położony na rozwiązanie w formie webowej.

10. Czy byłbyś/abyś zainteresowany przetestowaniem wersji demo systemu?

14 odpowiedzi



Rysunek 3.10: Wykres uzyskanych odpowiedzi na pytanie 10.

Wszyscy respondenci wyrazili chęć przetestowania wersji demo systemu, z czego aż 64,3% zadeklarowało się w sposób pewny.

11. Czy masz jakieś dodatkowe uwagi dot. systemu?

4 odpowiedzi

Warto zwrócić uwagę żeby synchronizacja danych z innych źródeł nie nadpisywała danych automatycznie, a wyświetlała wskazówkę dla użytkownika że takie dane się zmieniły i czy chce je zaakceptować

Jak to ADSPSi jest

Wg mnie powinna być sekcja z informacjami o danych urządzeniach na półce rack, oraz po wybraniu danej maszyny - powinna być swego rodzaju wizualizacja na mapce labu - podświetlony rack / jakaś ścieżka do niego

nie

Rysunek 3.11: Uzyskane odpowiedzi na pytanie 11.

Spośród czterech udzielonych odpowiedzi na pytanie otwarte dotyczące dodatkowych uwag do systemu (pytanie nieobowiązkowe), jedna (o treści "nie") nie zawiera informacji merytorycznych możliwych do analizy i została pominięta.

'Warto zwrócić uwagę żeby synchronizacja danych z innych źródłem nie nadpisywała danych automatycznie, a wyświetlała wskazówkę dla użytkownika że takie dane się zmieniły i czy chce je zaakceptować'

Odpowiedź ta wskazuje na potrzebę kontroli nad aktualizacją danych i mechanizmów jej walidacji. Zespół projektowy zwróci na to uwagę przy implementacji synchronizacji z innymi źródłami danych (np. pliki Excel), tworząc mechanizm, dzięki któremu nowe dane,

będą musiały zostać zaakceptowane przez osobę z odpowiednią rolą.

'Jak to ADSPSi jest'

Jeden z respondentów odniósł się do wewnętrznego narzędzia ADSPSi, wykorzystywanego do zarządzania sprzętem. System ten umożliwia jedynie podstawowe śledzenie lokalizacji hosta lub urządzenia (zapis jego położenia w formie tekstowej), jego parametrów technicznych oraz przypisania do zespołu lub właściciela. W opinii użytkowników ADSPSi nie zapewnia jednak wystarczającej funkcjonalności w zakresie zarządzania infrastrukturą laboratoryjną. Brakuje w nim m.in. możliwości wykonywania zdalnych akcji na urządzeniach, monitorowania ich dostępności w sieci czy automatycznego pobierania informacji o stanie sprzętu. Dane techniczne są często zapisywane w formie komentarzy, co utrudnia ich przetwarzanie i aktualizację. Dodatkowo, system jest postrzegany jako mało intuicyjny, z nieefektywnym interfejsem oraz długim czasem wyszukiwania informacji, co zniechęca użytkowników do regularnego aktualizowania danych.

'Wg mnie powinna być sekcja z informacjami o danych urządzeniach na półce rack, oraz po wybraniu danej maszyny - powinna być swego rodzaju wizualizacja na mapce labu - podświetlony rack / jakaś ścieżka do niego'

Respondent wskazał na potrzebę dodatkowej rozbudowy interfejsu wizualizującego / mapującego wygląd laboratorium (zrzut z góry). Oprócz możliwości zwizualizowania go poprzez wykorzystanie systemu drag'n'drop elementów (szafy rackowe, bench, alejki) i ich uporządkowanie, warto wprowadzić swego rodzaju ścieżkę prowadzącą do obecnie przeglądanej maszyny. Zespół projektowy również się z tym zgadza, że jest to dobry pomysł i postara się na wprowadzenie funkcjonalności "pokaż ścieżkę" - po wyborze tej opcji z menu danej maszyny, na mapie labu pojawiłaby się droga prowadząca od wejścia do labu - do racka w której znajduje się maszyna, w postaci żółtej linii.

3.3.6 Wnioski końcowe

Analiza odpowiedzi wskazuje na silną potrzebę stworzenie scentralizowanego systemu zarządzania laboratoriami / serwerowniami. Najważniejsze wymagania to:

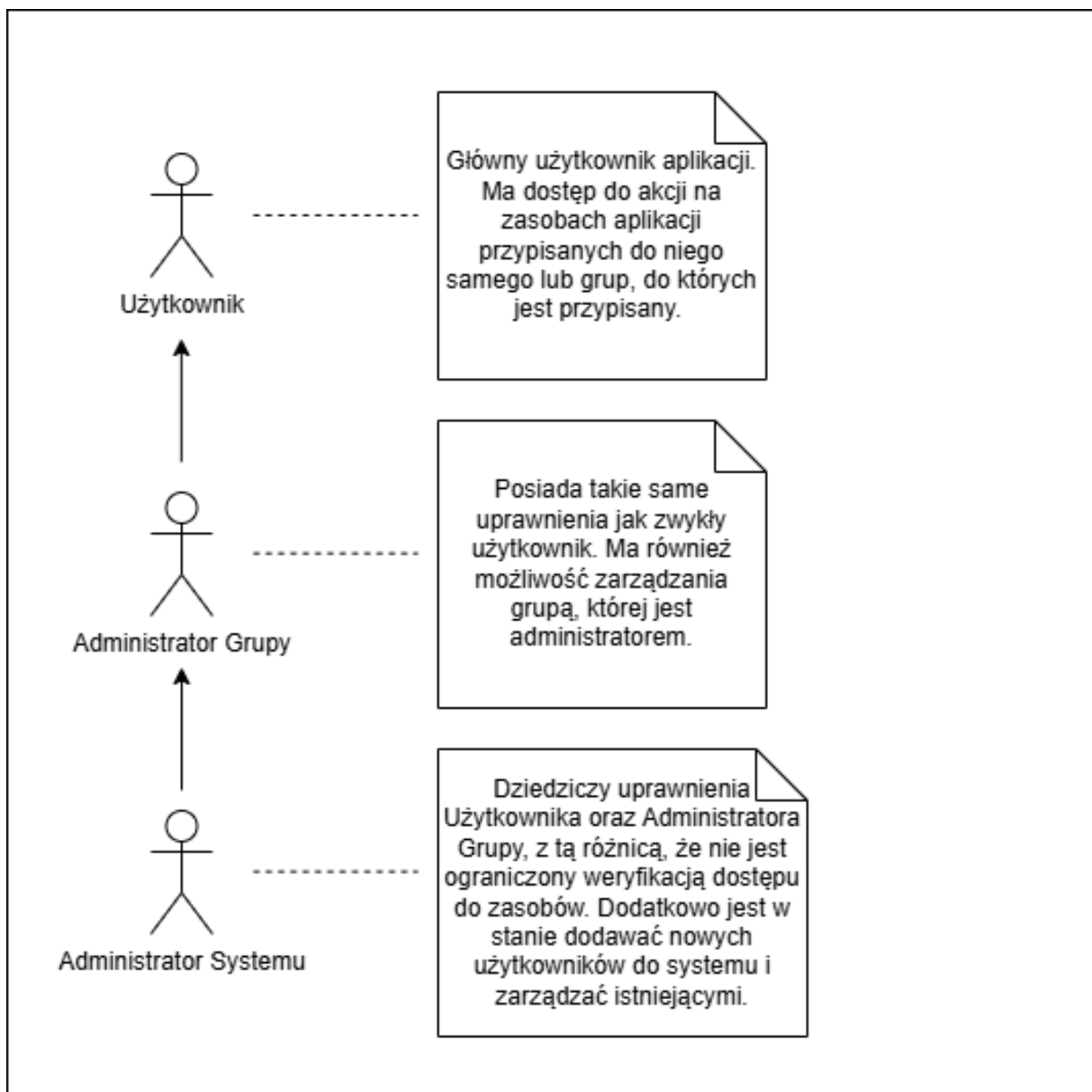
- Wizualne odwzorowanie laboratorium (71,4%)
- Statystyki i telemetria urządzeń (71,4%)
- Możliwość wykonywania zdalnych akcji (np. reboot, ping) (42,9%)
- Inwentarz materiałów eksploatacyjnych (57,1%)
- Możliwość przypisywania platform do konkretnych racków i półek (71,4%)
- Panel z podsumowaniem informacji o sprzęcie przypisanym do użytkownika (42,9%)

- Historia akcji i zmian sprzętu (78,6%)
- Panel zarządzania użytkownikami (role, uprawnienia) (42,9%)
- Wyszukiwarka sprzętu (78,6%)

System Labbyn powinien zostać zaprojektowany jako zintegrowane narzędzie, łączące ze sobą funkcję monitoringu maszyn, zarządzania maszynami / sprzętem, raportowania oraz inwentaryzacji w jednym, spójnym interfejsie.

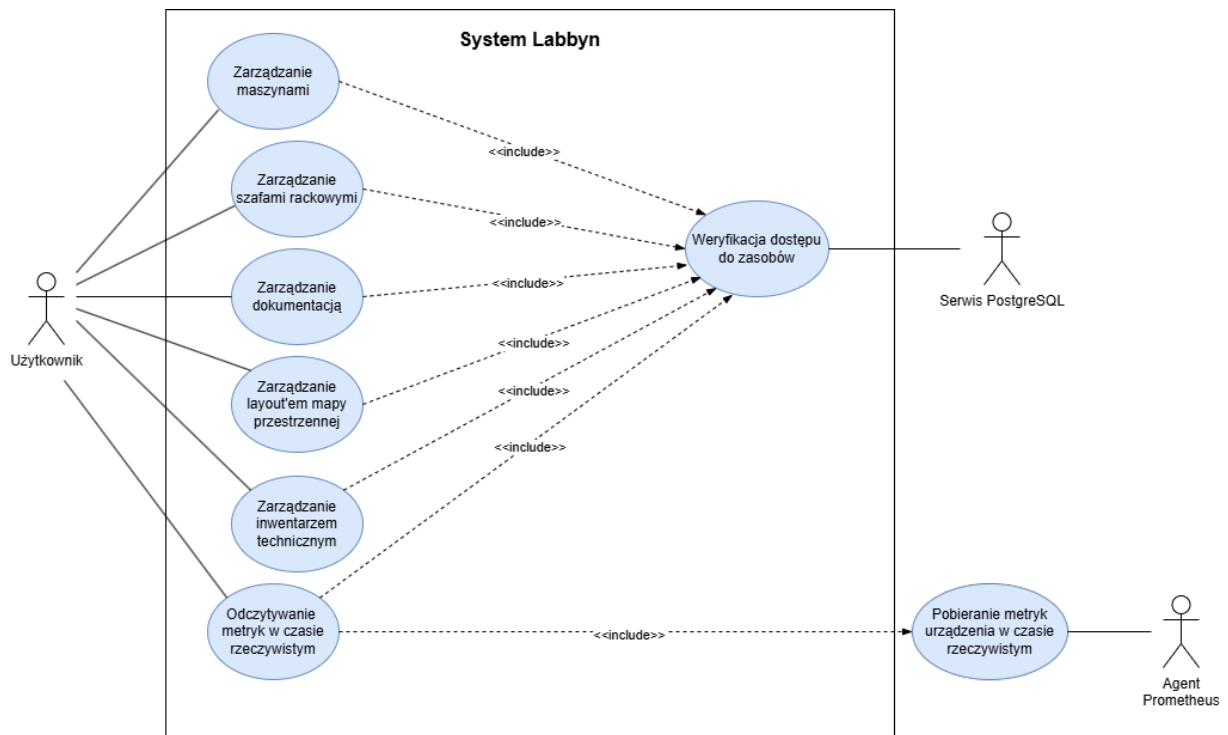
3.4 Diagramy

3.4.1 Główni Aktorzy

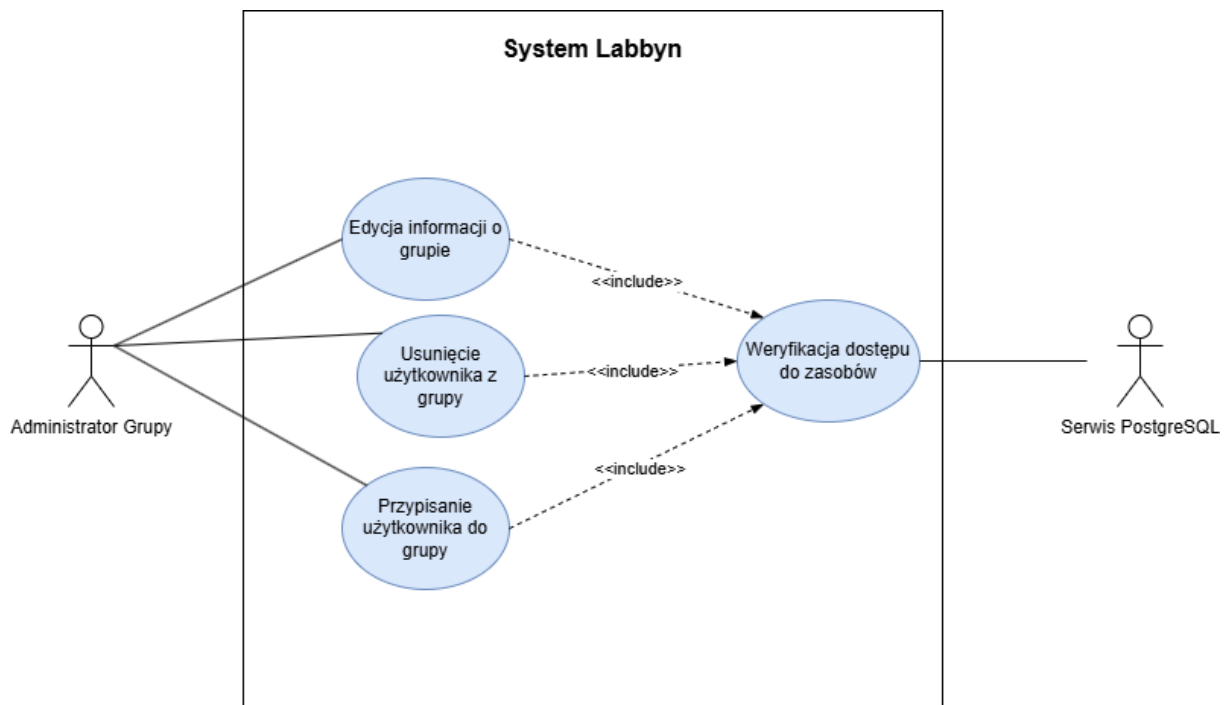


Rysunek 3.12: Diagram głównych aktorów

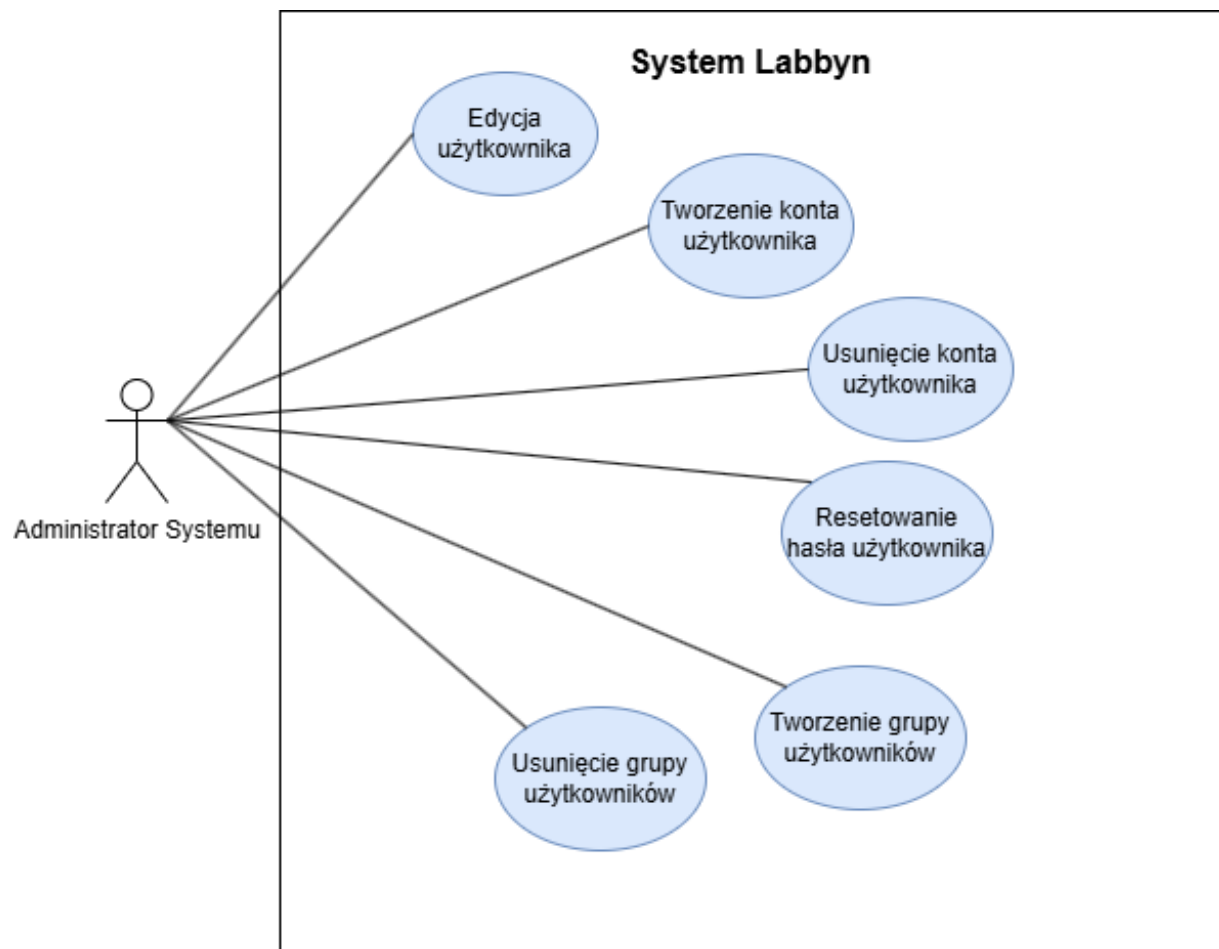
3.4.2 Diagramy przypadków użycia



Rysunek 3.13: Diagram przedstawiający przypadki użycia użytkownika

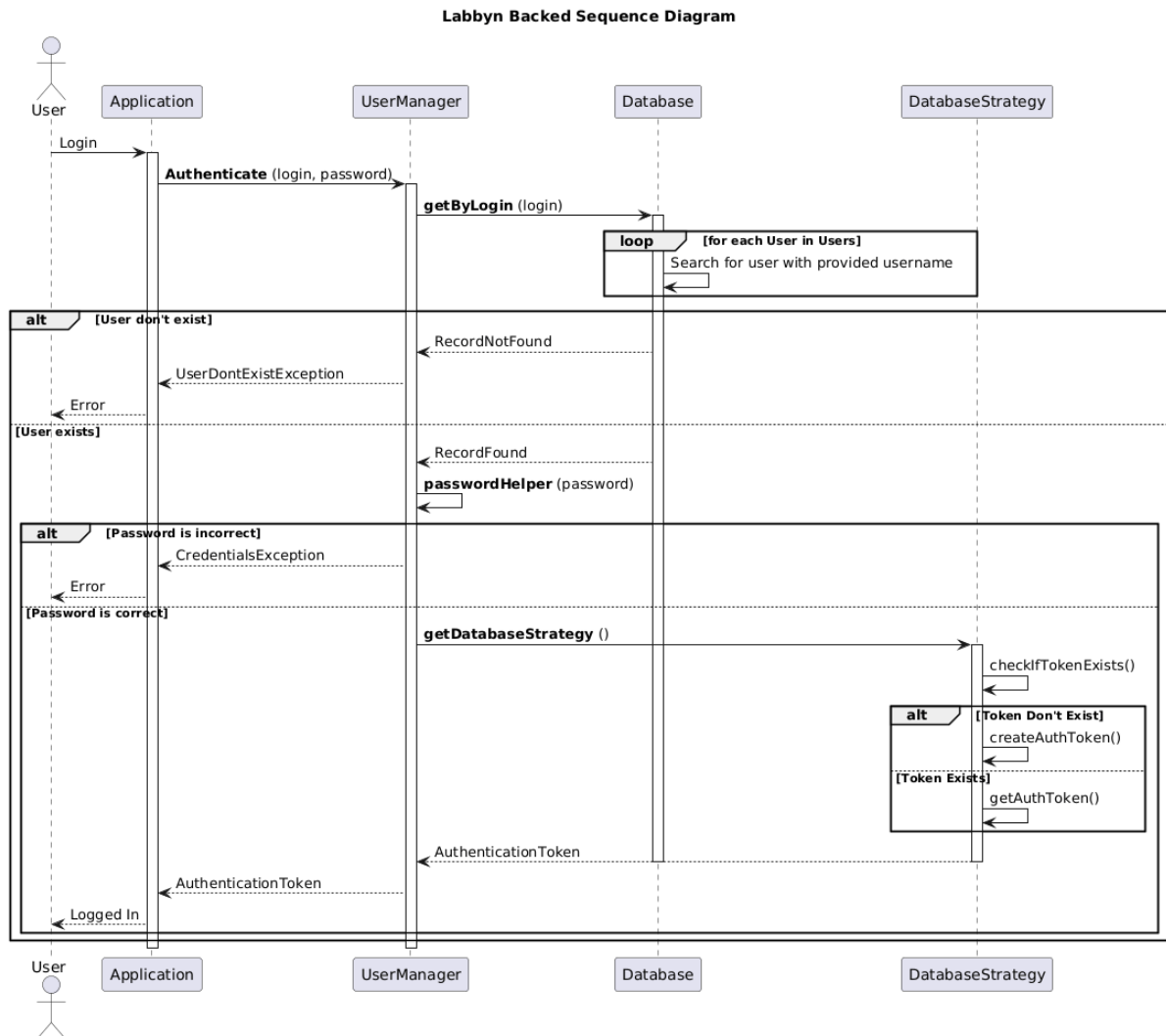


Rysunek 3.14: Diagram przedstawiający przypadki użycia administratora grupy

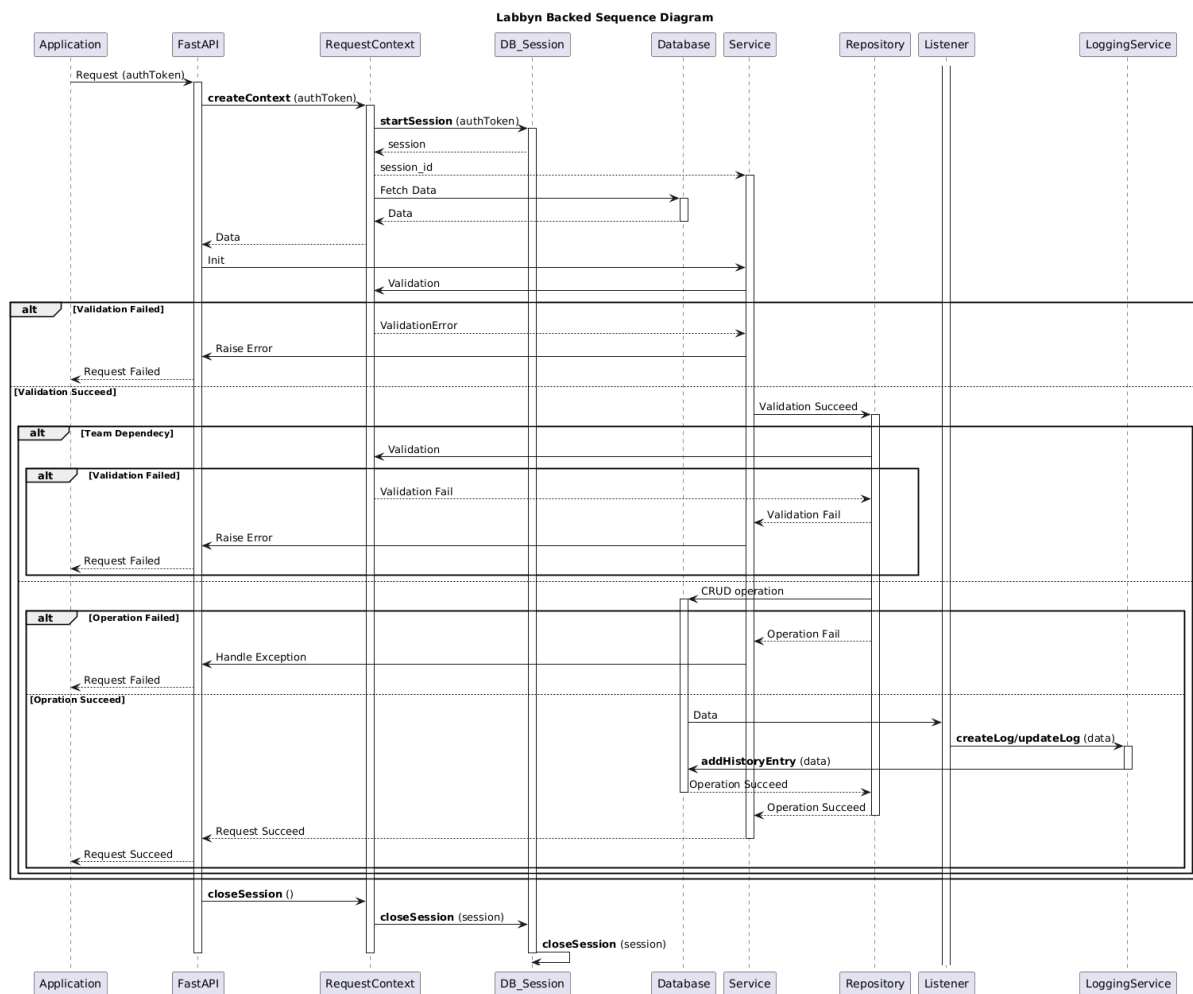


Rysunek 3.15: Diagram przedstawiający przypadki użycia administratora systemu

3.4.3 Diagramy sekwencji



Rysunek 3.16: Diagram sekwencji przedstawiający logowanie użytkownika



Rysunek 3.17: Diagram sekwencji przedstawiający przetwarzanie operacji CRUD

4 Inżynieria wymagań

4.1 Specyfikacja wymagań

Na podstawie wyników przeprowadzonej ankiety oraz wstępnych analiz, wyspecyfikowane zostały wymagania na system. Każdemu z wymagań przypisano priorytet, a w przypadku wymagań funkcjonalnych, podzielono je ze względu na odpowiedni moduł systemu.

Podczas cyklu wytwarzania oprogramowania dodane zostały wymagania funkcjonalne o akronimach od LAB-F-22 do LAB-F-45 oraz zmodyfikowane zostały następujące wymagania: LAB-F-04, LAB-F-06, LAB-F-08, LAB-F-09, LAB-F-16, LAB-F-17, LAB-F-18, LAB-F-19, LAB-F-20, LAB-I-01, LAB-NF-02. Zmiany te na celu miały zwiększenie zrozumienia architektury systemu z poziomu wymagań oraz doprecyzowanie szczegółów działania aplikacji. Wśród większych zmian wymienić można:

- LAB-I-01: Pierwotnie wymaganie zakładało, że system importować będzie dane z arkuszy kalkulacyjnych. Zespół jednomyślnie podjął jednak decyzję, że lepszym rozwiązaniem, które pozwoli dowieźć projekt w wyznaczonych ramach czasowych będzie obsługa tylko plików csv.
- LAB-NF-02: Wymaganie zakładało, że system będzie ograniczał ilość możliwych jednoczesnych połączeń używając parametru w konfiguracji aplikacji. Ze względów użytkowych parametr został usunięty a ilość jednoczesnych połączeń w systemie nie jest ograniczona żadnym mechanizmem.
- LAB-F-16: Pierwotnie wymaganie zakładało, że system będzie zapisywał zmiany w nim dokonane do dedykowanego pliku. Po przeprowadzonych wewnątrz zespołu dyskusjach, przeznaczeniem zmian stała się baza danych z dedykowaną tabelą.
- LAB-F-17: Na początku system zakładał 2 role użytkowników: użytkownik i administrator. Podczas procesu wytwarzania oprogramowania zespół napotkał problem związany z rolami użytkowników w systemie. Jako, że administrator grupy niekoniecznie powinien być administratorem systemu, zespół zdecydował się dodać dodatkową rolę w systemie - administrator grupy, który jest w stanie zarządzać grupą oraz jej zasobami jednocześnie nie mając uprawnień administratora systemu.

4.2 Udziałowcy

KARTA UDZIAŁOWCA	
Identyfikator:	UOB_01
Nazwa:	Zespół projektowy
Opis:	Zespół projektowy systemu Labbyn tworzący oprogramowanie
Typ udziałowca:	Ożywiony, bezpośredni
Punkt widzenia:	Techniczny, operator systemu
Ograniczenia:	Ograniczenia czasowe, ograniczenia budżetowe
Wymagania:	LAB-D-01, LAB-D-02, LAB-D-03, LAB-D-04, LAB-D-05, LAB-D-06, LAB-E-01, LAB-E-02, LAB-E-03, LAB-F-01, LAB-F-02, LAB-F-03, LAB-F-04, LAB-F-05, LAB-F-06, LAB-F-07, LAB-F-08, LAB-F-09, LAB-F-10, LAB-F-11, LAB-F-12, LAB-F-13, LAB-F-14, LAB-F-15, LAB-F-16, LAB-F-17, LAB-F-18, LAB-F-19, LAB-F-20, LAB-F-21, LAB-F-22, LAB-F-23, LAB-F-24, LAB-F-25, LAB-F-26, LAB-F-27, LAB-F-28, LAB-F-29, LAB-F-30, LAB-F-31, LAB-F-32, LAB-F-33, LAB-F-34, LAB-F-35, LAB-F-36, LAB-F-37, LAB-F-38, LAB-F-39, LAB-F-40, LAB-F-41, LAB-F-42, LAB-F-43, LAB-F-44, LAB-F-45, LAB-I-01, LAB-I-02, LAB-I-03, LAB-I-04, LAB-NF-01, LAB-NF-02, LAB-NF-03, LAB-NF-04, LAB-NF-05, LAB-NF-06, LAB-NF-07, LAB-NF-08

KARTA UDZIAŁOWCA	
Identyfikator:	UOB_02
Nazwa:	Opiekun projektu
Opis:	Promotor projektu inżynierskiego
Typ udziałowca:	Ożywiony, bezpośredni
Punkt widzenia:	Biznesowy
Ograniczenia:	Brak wpływu na tworzony kod
Wymagania:	Brak

KARTA UDZIAŁOWCA	
Identyfikator:	UOB_03
Nazwa:	Użytkownicy systemu
Opis:	Docelowy użytkownik aplikacji
Typ udziałowca:	Ożywiony, bezpośredni
Punkt widzenia:	Operator systemu
Ograniczenia:	
Wymagania:	Brak

KARTA UDZIAŁOWCA	
Identyfikator:	UOB_04
Nazwa:	Zarząd firmy
Opis:	Osoby na stanowiskach managerskich w organizacji
Typ udziałowca:	Ożywiony, bezpośredni
Punkt widzenia:	Biznesowy
Ograniczenia:	
Wymagania:	Brak

4.3 Wymagania ogólne i dziedzinowe

KARTA WYMAGANIA			
Identyfikator:	LAB-D-01	Priorytet:	M
Nazwa:	Kontrola dostępu		
Opis:	System musi być dostępny tylko dla użytkowników posiadających konto w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-D-02	Priorytet:	M
Nazwa:	Zarządzanie laboratorium		
Opis:	System musi wspierać zarządzanie laboratorium IT		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-D-03	Priorytet:	M
Nazwa:	Inwentarz Techniczny		
Opis:	System musi wspierać zarządzanie inwentarzem technicznym		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-D-04	Priorytet:	M
Nazwa:	Monitorowanie stanu		
Opis:	System musi wspierać monitorowanie stanu urządzeń		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-D-05	Priorytet:	M
Nazwa:	Mapa przestrzenna laboratorium		
Opis:	System musi umożliwiać odwzorowanie fizycznego układu laboratorium		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-D-06	Priorytet:	M
Nazwa:	Ochrona danych		
Opis:	System musi spełniać podstawowe wymogi ochrony danych		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.4 Wymagania Funkcjonalne

4.4.1 Zakładka 'Labs'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-01	Priorytet:	M
Nazwa:	System musi posiadać widok całego laboratorium		
Opis:	Jako pracownik związany ze sprzętem w laboratorium/serwerowni chcę mieć możliwość sprawdzenia układu pomieszczenia w celu efektywnego planowania przyszłych i bieżących zadań		
Kryteria akceptacji:	System posiada w pełni funkcjonalną zakładkę „Map” dla każdego laboratorium		
Dane wejściowe:	Lab z mapą przestrzenną, użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik jest zalogowany do systemu i widzi zakładkę „Labs”, a wewnątrz niej przynajmniej 1 Lab		
Warunki końcowe:	Mapa przestrzenna laboratorium jest widoczna i w pełni funkcjonalna		
Sytuacje wyjątkowe:	Lab nie posiada mapy laboratorium		
Szczegóły implementacji:	Dodanie przycisku „Show Map” do każdego obiektu laboratorium.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-14		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-14	Priorytet:	M
Nazwa:	System musi posiadać system Drag n' Drop elementów laboratorium		
Opis:	Jako użytkownik systemu, chcę móc ustawić elementy interaktywne na mapie przestrzennej laboratorium, żeby odwzorować stan faktyczny pomieszczenia.		
Kryteria akceptacji:	Mapa przestrzenna laboratorium posiada w pełni funkcjonalny system Drag n' Drop z możliwością aranżacji elementów na dostępnej powierzchni.		
Dane wejściowe:	Lab z mapą przestrzenną, użytkownik systemu z dowolną przypisaną rolą z uprawnieniami do wybranego laboratorium		
Warunki początkowe:	użytkownik jest zalogowany do systemu, w zakładce „Labs” widzi przynajmniej 1 lab do którego ma uprawnienia edycji z przyciskiem „Show Map”, po kliknięciu na ten przycisk przeniesiony jest do widoku przestrzennego laboratorium.		
Warunki końcowe:	użytkownik jest w stanie rozmieszczać elementy laboratorium w dowolny sposób z możliwością zapisu zmian.		
Sytuacje wyjątkowe:	użytkownik nie posiada praw edycji laboratorium na mapie przestrzennej.		
Szczegóły implementacji:	Dodanie przycisku „Edit” możliwego do kliknięcia w momencie gdy zalogowany użytkownik jest przypisany do danego laboratorium.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-01		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-22	Priorytet:	M
Nazwa:	System musi posiadać panel dostępnych laboratoriów		
Opis:	Jako użytkownik systemu, chcę mieć łatwy dostęp do informacji o wszystkich dostępnych laboratoriach w systemie.		
Kryteria akceptacji:	Zaimplementowana zakładka "Labs"		
Dane wejściowe:	Użytkownik z dowolną przypisaną rolą z uprawnieniami do wybranego laboratorium		
Warunki początkowe:	Użytkownik jest zalogowany do systemu.		
Warunki końcowe:	Użytkownik w zakładce "Labs" widzi przynajmniej 1 laboratorium		
Sytuacje wyjątkowe:	Brak utworzonych laboratoriów w systemie		
Szczegóły implementacji:	Implementacja zakładki "Labs"		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-01, LAB-F-14, LAB-F-23		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-22	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów laboratorium		
Opis:	Jako użytkownik systemu, chcę widzieć szczegóły laboratorium w dedykowanym do tego panelu.		
Kryteria akceptacji:	Implementacja panelu szczegółów laboratorium		
Dane wejściowe:	Użytkownik z dowolną rolą oraz przynajmniej 1 dostępnym laboratorium w zakładce "Labs"		
Warunki początkowe:	Użytkownik jest zalogowany do systemu, jest w stanie kliknąć w wybrane laboratorium		
Warunki końcowe:	Użytkownik zostaje przeniesiony do panelu szczegółów wybranego laboratorium.		
Sytuacje wyjątkowe:	Brak utworzonych laboratoriów w systemie		
Szczegóły implementacji:	Implementacja widoku szczegółowego laboratorium		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-01, LAB-F-14, LAB-F-22, LAB-F-38		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-38	Priorytet:	M
Nazwa:	System musi zezwalać na dodawanie nowych laboratoriów oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik systemu chcę mieć możliwość dodawania nowych laboratoriów w systemie oraz edycję istniejących		
Kryteria akceptacji:	System z zaimplementowanym formularzem dodawania nowego laboratorium oraz mechanizmem edycji laboratorium z poziomu widoku szczegółowego.		
Dane wejściowe:	Użytkownik systemu z dowolną rolą		
Warunki początkowe:	Użytkownik zalogowany, w panelu bocznym widzi zakładkę dodania laboratorium		
Warunki końcowe:	Po kliknięciu w zakładkę dodania maszyny użytkownik widzi formularz dodania laboratorium do systemu. Po wypełnieniu wszystkich pól i kliknięciu przycisku zatwierdzenia widzi dodane laboratorium w systemie. Po wejściu w panel szczegółów dodanego laboratorium użytkownik widzi przycisk edycji po kliknięciu w który panel zmienia tryb podglądu w tryb edycji.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie formularza dodawania laboratorium oraz trybu edycji w panelu szczegółów laboratorium.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-23		

4.4.2 Panel Urządzenia

KARTA WYMAGANIA			
Identyfikator:	LAB-F-02	Priorytet:	M
Nazwa:	System musi pokazywać statystyki urządzeń		
Opis:	Jako użytkownik systemu, chcę mieć możliwość odczytania statystyk każdego urządzenia w jego unikalnej zakładce z informacjami		
Kryteria akceptacji:	Każde urządzenie posiada widok ze szczegółowymi statystykami		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik jest zalogowany do systemu, w zakładce „Labs” widzi przynajmniej 1 laboratorium z przypisanym przynajmniej 1 urządzeniem, który może kliknąć		
Warunki końcowe:	użytkownik jest w stanie przenieść się na widok ze szczegółami urządzenia po kliknięciu kafelka z nazwą urządzenia		
Sytuacje wyjątkowe:	Brak dostępnych urządzeń w systemie		
Szczegóły implementacji:	Po kliknięciu na obiekt maszyny, aktywowanie okna urządzenia ze statystykami		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-03, LAB-F-05, LAB-F-10, LAB-F-11, LAB-F-15		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-03	Priorytet:	C
Nazwa:	System powinien pozwolić na wykonywanie zdalnych akcji np. Reboot, ping		
Opis:	Jak użytkownik systemu chce mieć dostęp do wykonywania zdalnych komend na maszynie z poziomu widoku maszyny		
Kryteria akceptacji:	System posiada możliwość wykonania zdalnej komendy z poziomu interfejsu użytkownika		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik jest zalogowany do systemu, na ekranie szczegółów urządzenia widoczny jest panel do wpisania zdalnej komendy		
Warunki końcowe:	użytkownik jest w stanie wysłać komendę zdalnie na urządzenie z poziomu systemu		
Sytuacje wyjątkowe:	Wybrane urządzenie nie jest podłączone do sieci		
Szczegóły implementacji:	Na ekranie szczegółów urządzenia dodanie sekcji do wysyłania zdalnego polecenia.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-02, LAB-F-05, LAB-F-10, LAB-F-11, LAB-F-15		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-05	Priorytet:	M
Nazwa:	System musi zezwolić na przypisywanie platform do konkretnych racków i półek		
Opis:	Jako użytkownik chcę mieć możliwość przypisania platformy do wybranej przeze mnie półki w celu skutecznego wyszukiwania urządzeń znajdujących się na tej samej półce lub racku.		
Kryteria akceptacji:	System posiada możliwość przypisania platformy do jednego z racków lub półek.		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, labem oraz półką lub rackiem, użytkownik systemu z dowolną przypisaną rolą z przynajmniej 1 przypisaną do niego lub jego grupy maszyną		
Warunki początkowe:	użytkownik jest zalogowany do systemu, na ekranie szczegółów urządzenia widoczny jest aktualnie przypisany rack lub półka		
Warunki końcowe:	użytkownik jest w stanie zmienić przypisanie urządzenia do innego reacku lub półki		
Sytuacje wyjątkowe:	Brak innych racków lub półek w systemie		
Szczegóły implementacji:	Na ekranie szczegółów urządzenia dodanie sekcji z przypisanym rackiem lub półką		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-02, LAB-F-03, LAB-F-10, LAB-F-11, LAB-F-15		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-10	Priorytet:	W
Nazwa:	System powinien posiadać odnośnik do informacji o danym sprzęcie/ layout'u PCB		
Opis:	Jako użytkownik chcę mieć możliwość przejść do karty produktu ze szczegółami technicznymi lub dotyczącymi PCB z poziomu systemu		
Kryteria akceptacji:	System posiada odnośnik do strony z dokumentacją techniczną sprzętu		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	Brak, wymaganie odrzucone		
Warunki końcowe:	Brak, wymaganie odrzucone		
Sytuacje wyjątkowe:	Brak, wymaganie odrzucone		
Szczegóły implementacji:	Brak, wymaganie odrzucone		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-02, LAB-F-03, LAB-F-05, LAB-F-11, LAB-F-15		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-11	Priorytet:	M
Nazwa:	Dane w widoku urządzenia		
Opis:	Jako użytkownik systemu, chcę mieć dostęp do następujących danych na poziomie widoku urządzenia: Lokalizacja, Nazwa Hosta, Adres IP oraz MAC, Status w sieci, Port w PDU, Tag (przypisany zespół / przeznaczony zespół), System Operacyjny, Informacje o sprzęcie (CPU, Dysk, RAM), Numer Seryjny		
Kryteria akceptacji:	Wszystkie pola zawarte w wymaganiu są dostępne w systemie w widoku urządzenia		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, użytkownik systemu z dowolną przypisaną rolą z przynajmniej 1 przypisaną do niego lub jego grupy maszyną		
Warunki początkowe:	użytkownik jest zalogowany do systemu, na ekranie szczegółów urządzenia		
Warunki końcowe:	użytkownik jest w stanie zobaczyć wszystkie dane lub miejsce na ich umieszczenie w systemie na poziomie widoku urządzenia		
Sytuacje wyjątkowe:	Brak dostępnych lub przypisanych do użytkownika urządzeń w systemie		
Szczegóły implementacji:	Na ekranie szczegółów urządzenia dodać pola wypisane w wymaganiu.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-02, LAB-F-03, LAB-F-05, LAB-F-10, LAB-F-15		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-15	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów urządzenia		
Opis:	Jako użytkownik systemu chcę mieć dostęp do szczegółów urządzenia z poziomu systemu		
Kryteria akceptacji:	Zaimplementowany panel szczegółów urządzenia		
Dane wejściowe:	System z przynajmniej 1 dostępnym urządzeniem, użytkownik systemu z dowolną przypisaną rolą z przynajmniej 1 przypisaną do niego lub jego grupy maszyną		
Warunki początkowe:	Użytkownik jest zalogowany do systemu		
Warunki końcowe:	Użytkownik jest w stanie przejść do widoku ze szczegółami urządzenia z poziomu systemu		
Sytuacje wyjątkowe:	Brak dostępnych urządzeń w systemie		
Szczegóły implementacji:	Dodanie widoku urządzenia po kliknięciu w kafelek z nazwą urządzenia z poziomu mapy przestrzennej lub paska wyszukiwania		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-02, LAB-F-03, LAB-F-05, LAB-F-10, LAB-F-11, LAB-F-36		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-24	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów szafy rackowej		
Opis:	Jako użytkownik systemu, chcę mieć możliwość sprawdzenia szczegółów danej szafy rackowej z poziomu systemu.		
Kryteria akceptacji:	Zaimplementowany widok szczegółowy szafy rackowej		
Dane wejściowe:	Użytkownik z dowolną rolą, przynajmniej 1 szafa rackowa w systemie, do której użytkownik ma dostęp.		
Warunki początkowe:	Użytkownik zalogowany do systemu, na poziomie widoku szczegółów urządzenia widzi rubrykę "Rack", jest w stanie kliknąć w odnośnik		
Warunki końcowe:	Użytkownik zostaje przeniesiony na panel szczegółów szafy rackowej		
Sytuacje wyjątkowe:	Brak szaf rackowych przypisanych do użytkownika lub jego grupy.		
Szczegóły implementacji:	Implementacja panelu szczegółów szafy rackowej		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-37		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-36	Priorytet:	M
Nazwa:	System musi zezwalać na dodawanie nowych maszyn oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik systemu chcę mieć możliwość dodawania nowych maszyn w systemie oraz edycję istniejących		
Kryteria akceptacji:	System z zaimplementowanym formularzem dodawania nowej maszyny oraz mechanizmem edycji maszyny z poziomu widoku szczegółowego maszyny.		
Dane wejściowe:	Użytkownik systemu z dowolną rolą		
Warunki początkowe:	Użytkownik zalogowany, w panelu bocznym widzi zakładkę dodania maszyny		
Warunki końcowe:	Po kliknięciu w zakładkę dodania maszyny użytkownik widzi formularz dodania maszyny do systemu. Po wypełnieniu wszystkich pól i kliknięciu przycisku zatwierdzenia widzi dodaną maszynę w systemie. Po wejściu w panel szczegółów dodanej maszyny użytkownik widzi przycisk edycji po kliknięciu w który panel zmienia tryb podglądu w tryb edycji.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie formularza dodawania maszyny oraz trybu edycji w panelu szczegółów maszyny.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-15, LAB-F-39, LAB-F-40		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-37	Priorytet:	M
Nazwa:	System musi zezwalać na dodawanie nowych szaf rackowych oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik systemu chcę mieć możliwość dodawania nowych racków w systemie oraz edycję istniejących		
Kryteria akceptacji:	System z zaimplementowanym formularzem dodawania nowego racka oraz mechanizmem edycji racka z poziomu widoku szczegółowego racku.		
Dane wejściowe:	Użytkownik systemu z dowolną rolą		
Warunki początkowe:	Użytkownik zalogowany, w panelu bocznym widzi zakładkę dodania racku		
Warunki końcowe:	Po kliknięciu w zakładkę dodania racka użytkownik widzi formularz dodania racka do systemu. Po wypełnieniu wszystkich pól i kliknięciu przycisku zatwierdzenia widzi dodany rack w systemie. Po wejściu w panel szczegółów dodanego racka użytkownik widzi przycisk edycji po kliknięciu w który panel zmienia tryb podglądu w tryb edycji.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie formularza dodawania racka oraz trybu edycji w panelu szczegółów racka.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-24		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-39	Priorytet:	M
Nazwa:	System musi posiadać funkcję 'Auto-Discovery', która pobierze aktualną konfigurację sprzętową maszyny		
Opis:	Jako użytkownik systemu chcę mieć możliwość pobrania konfiguracji sprzętowej bezpośrednio z maszyny do systemu co skróci czas dodawania nowych maszyn		
Kryteria akceptacji:	System z zaimplementowaną funkcją 'Auto-Discovery'		
Dane wejściowe:	Użytkownik systemu z dowolną rolą		
Warunki początkowe:	Użytkownik w formularzu dodawania nowej maszyny		
Warunki końcowe:	Podczas dodawania nowej maszyny użytkownik widzi opcję 'Auto-Discovery' która podczas dodawania maszyny wypełni pola szczegółów urządzenia.		
Sytuacje wyjątkowe:	Operacja nie powiodła się - powiadomienie o błędzie		
Szczegóły implementacji:	Zaimplementowanie mechanizmu 'Auto-Discovery' w formularzu dodawania maszyny		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-36		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-40	Priorytet:	M
Nazwa:	System musi zezwalać na aktualizację danych maszyny przy użyciu funkcji 'Auto-Discovery'		
Opis:	Jako użytkownik systemu chcę mieć możliwość pobrania konfiguracji sprzętowej bezpośrednio z maszyny do systemu co skróci czas dodawania nowych maszyn oraz aktualizacji informacji o maszynie po zmianach konfiguracji		
Kryteria akceptacji:	System z zaimplementowaną funkcją 'Auto-Discovery'		
Dane wejściowe:	Użytkownik systemu z dowolną rolą, przynajmniej 1 maszyna dodana do systemu.		
Warunki początkowe:	Użytkownik w panelu szczegółów maszyny widzi przycisk 'Auto-Discovery'		
Warunki końcowe:	Po kliknięciu użytkownik widzi komunikat o powodzeniu operacji i zmiany w szczegółach urządzenia.		
Sytuacje wyjątkowe:	Operacja nie powiodła się - powiadomienie o błędzie		
Szczegóły implementacji:	Zaimplementowanie mechanizmu 'Auto-Discovery'		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-36		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-41	Priorytet:	M
Nazwa:	System musi zezwalać na przypisanie szaf rackowych, maszyn oraz materiałów eksploatacyjnych do istniejącej grupy		
Opis:	Jako użytkownik systemu chce mieć możliwość przypisywania elementów roboczych do konkretnych grup aby łatwo zidentyfikować zasoby każdego z zespołów		
Kryteria akceptacji:	System z zaimplementowanym mechanizmem przypisywania elementów roboczych do grup		
Dane wejściowe:	Przynajmniej 1 dostępna grupa użytkowników oraz przynajmniej 1 użytkownik przypisany do tej grupy w systemie		
Warunki początkowe:	Użytkownik z poziomu dodawania lub edycji elementu		
Warunki końcowe:	Użytkownik może przypisać element do jednej ze swoich grup lub usunąć przypisanie		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja systemu przypisywania elementów w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:			

4.4.3 Zakładka 'Inventory'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-04	Priorytet:	S
Nazwa:	System musi posiadać panel materiałów eksploatacyjnych		
Opis:	Jako użytkownik systemu chcę mieć dostęp do inwentarza materiałów eksploatacyjnych aby szybko i efektywnie wyszukiwać niezbędne do mojej pracy narzędzia i kontrolować ich ilość		
Kryteria akceptacji:	Zakładka „Inventory” zaimplementowana w systemie		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik jest zalogowany do systemu		
Warunki końcowe:	użytkownik widzi w pasku bocznym zakładkę „Inventory”, po której kliknięciu przenoszony jest na ekran inwentarza		
Sytuacje wyjątkowe:	Brak paska bocznego		
Szczegóły implementacji:	Dodanie zakładki „Inventory” w paski bocznym		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-12		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-12	Priorytet:	M
Nazwa:	Informacje w inwentarzu technicznym		
Opis:	Jako użytkownik systemu, chcę mieć dostęp do następujących danych na poziomie inwentarza technicznego: Nazwa i nr seryjny, Dostępność, przypisanie do konkretnych maszyn, przypisanie do konkretnych zespołów, typ urządzenia/kategorie, Historia wypożyczeń, lokalizacja		
Kryteria akceptacji:	Wszystkie pola zawarte w wymaganiu są dostępne w systemie w inwentarzu technicznym		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą, przynajmniej 1 element w inwentarzu przypisany do użytkownika lub jego grupy		
Warunki początkowe:	użytkownik zalogowany do systemu na zakładce „Inventory”		
Warunki końcowe:	użytkownik jest w stanie zobaczyć wszystkie dane lub miejsce na ich umieszczenie w systemie na poziomie inwentarzu		
Sytuacje wyjątkowe:	Brak przypisanych elementów do użytkownika w inwentarzu		
Szczegóły implementacji:	Do każdego elementu inwentarzu dodać pola wypisane w wymaganiu.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-04		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-42	Priorytet:	M
Nazwa:	System musi posiadać system wypożyczeń materiałów eksploatacyjnych		
Opis:	Jako użytkownik systemu chcę mieć możliwość wypożyczenia materiałów eksploatacyjnych innemu zespołowi co usprawni identyfikację lokalizacji sprzętu		
Kryteria akceptacji:	System z zaimplementowanym mechanizmem wypożyczeń		
Dane wejściowe:	Użytkownik systemu z dowolną rolą, przynajmniej 1 element inwentarzu technicznego przypisany do użytkownika, dodatkowy użytkownik w systemie.		
Warunki początkowe:	Użytkownik zalogowany do systemu w panelu materiałów eksploatacyjnych widzi przynajmniej 1 wpis w liście inwentarzu		
Warunki końcowe:	Po kliknięciu w akcje na elemencie, użytkownik widzi przycisk wypożyczenia, po kliknięciu w który widzi formularz wypożyczenia. Po wypełnieniu wszystkich pól i kliknięciu przycisku akceptacji element zostaje wypożyczony i przypisany tymczasowo do docelowego użytkownika.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja mechanizmu i formularza wypożyczeń		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:			

KARTA WYMAGANIA			
Identyfikator:	LAB-F-43	Priorytet:	M
Nazwa:	System musi zezwalać na przypisanie materiałów eksploatacyjnych do istniejącej maszyny, laboratorium i szafy rackowej		
Opis:	Jako użytkownik systemu chce mieć możliwość przypisywania elementów inwentarzu technicznego do konkretnej maszyny, laboratorium lub szafy rackowej by ułatwić zarządzanie dostępnym sprzętem.		
Kryteria akceptacji:	System z zaimplementowanym mechanizmem przypisywania elementów inwentarzu technicznego do elementów roboczych.		
Dane wejściowe:	Użytkownik systemu z dowolną przypisaną rolą, przynajmniej 1 element inwentarzu przypisany do użytkownika lub jego grupy, przynajmniej 1 laboratorium, maszyna lub rack w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, na poziomie inwentarzu technicznego		
Warunki końcowe:	Użytkownik może przypisać element do laboratorium, maszyny lub racka		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja systemu przypisywania elementów inwentarzu w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:			

4.4.4 Zakładka 'Dashboard'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-06	Priorytet:	S
Nazwa:	System powinien posiadać panel z podsumowaniem informacji o sprzęcie przypisanym do użytkownika oraz jego zespołów		
Opis:	Jako użytkownik systemu chcę mieć dostęp do panelu z podsumowaniem elementów przypisanych do mnie lub mojego zespołu, co pozwoli skrócić czas jego wyszukiwania		
Kryteria akceptacji:	Zakładka „Dashboard” widoczna z poziomu systemu na pasku bocznym		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik zalogowany do systemu		
Warunki końcowe:	użytkownik widzi zakładkę „Dashboard” na pasku bocznym, po której kliknięciu przenoszony jest na ekran z podsumowaniem		
Sytuacje wyjątkowe:	użytkownik nie ma przypisanego żadnego elementu		
Szczegóły implementacji:	Dodanie zakładki „Dashboard” na pasku bocznym		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.4.5 Search Bar (Pasek wyszukiwania)

KARTA WYMAGANIA			
Identyfikator:	LAB-F-07	Priorytet:	M
Nazwa:	System musi posiadać wyszukiwarke sprzętu		
Opis:	Jak użytkownik systemu chce mieć dostęp do paska wyszukiwania by przyspieszyć identyfikowanie sprzętu		
Kryteria akceptacji:	Pasek wyszukiwania zaimplementowany w systemie i dodany do paska bocznego		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	użytkownik zalogowany do systemu		
Warunki końcowe:	użytkownik widzi pasek wyszukiwania na pasku bocznym, w którym jest w stanie wpisać wyszukiwaną frazę		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie paska wyszukiwania na pasku bocznym		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.4.6 Zakładka 'History'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-08	Priorytet:	M
Nazwa:	System musi posiadać panel historii operacji na sprzęcie		
Opis:	Jako użytkownik chcę śledzić zmiany sprzętu by efektywnie identyfikować modyfikacje w jego konfiguracji		
Kryteria akceptacji:	Mechanizm zapisywania historii akcji zaimplementowany w systemie		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą i przynajmniej 1 przypisanym elementem do niego lub jego grupy		
Warunki początkowe:	użytkownik zalogowany do systemu na ekranie wybranego elementu		
Warunki końcowe:	użytkownik wykonuje zmianę, która jest wykryta przez system		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie mechanizmu zapisywania wykonywanych akcji w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-16, LAB-F-21		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-16	Priorytet:	C
Nazwa:	System powinien rejestrować operacje użytkownika na danych i zapisywać je w bazie danych		
Opis:	Jako użytkownik systemu chcę by zmiany wykonywane w systemie zapisywane były w bazie danych by zapewnić trwałość danych w kopiach zapasowych systemu		
Kryteria akceptacji:	Zmiany logowane przez system zapisywane są do bazy danych		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą i przynajmniej 1 przypisanym elementem do niego lub jego grupy		
Warunki początkowe:	użytkownik zalogowany do systemu na ekranie wybranego elementu jest w stanie wykonać modyfikację		
Warunki końcowe:	Zmiana wykryta przez system, zapisana w bazie danych		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie funkcjonalności zapisu historii do pliku		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-08, LAB-F-21		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-21	Priorytet:	C
Nazwa:	Wersjonowanie historii		
Opis:	Jako użytkownik systemu chcę mieć możliwość wersjonowania historii zmian żeby w prosty sposób sprawdzić stan przed modyfikacją		
Kryteria akceptacji:	Funkcjonalność wersjonowania historii zaimplementowana w systemie z możliwością wyboru wersji		
Dane wejściowe:	użytkownik systemu z dowolną przypisaną rolą i przynajmniej 1 przypisanym elementem do niego lub jego grupy, na którym wykonane zostały modyfikacje		
Warunki początkowe:	użytkownik z poziomu widoku wybranego elementu widzi pasek wyboru z opisem wersji		
Warunki końcowe:	użytkownik jest w stanie wybrać wcześniejszą wersję widoku z przed zmian		
Sytuacje wyjątkowe:	Brak zapisanych zmian w elemencie		
Szczegóły implementacji:	Dodanie funkcjonalności zapisu historii do bazy danych		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-08, LAB-F-21, LAB-F-25		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-25	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów zmiany w systemie		
Opis:	Jako użytkownik chcę mieć możliwość dokładnej analizy zmian w systemie.		
Kryteria akceptacji:	Zaimplementowany panel szczegółów zmian		
Dane wejściowe:	Użytkownik systemu z dowolną przypisaną rolą, przynajmniej 1 dokonana zmiana w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, w zakładce "History" widzi zmiany dokonane w systemie, jest w stanie kliknąć w każdą z nich.		
Warunki końcowe:	Po kliknięciu użytkownik widzi panel szczegółów zmiany		
Sytuacje wyjątkowe:	Brak zmian w systemie		
Szczegóły implementacji:	Implementacja panelu szczegółów zmian		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-08, LAB-F-16, LAB-F-21		

4.4.7 Zakładka 'Users'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-09	Priorytet:	M
Nazwa:	Panel administratora musi posiadać panel zarządzania użytkownikami systemu		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chcę mieć dostęp do panelu, z poziomu którego będę w stanie zarządzać użytkownikami systemu		
Kryteria akceptacji:	Panel administratora z zaimplementowanym panelem zarządzania użytkownikami		
Dane wejściowe:	Użytkownik systemu z przypisaną rolą administratora, dodatkowy użytkownik w systemie z dowolną rolą, przynajmniej 1 dodatkowy użytkownik w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, w pasku bocznym widzi zakładkę "Admin" poniżej której znajdują się moduły administracyjne spośród których widoczny jest moduł "Users"		
Warunki końcowe:	Po kliknięciu w moduł "Users" użytkownik przenoszony jest do panelu zarządzania użytkownikami.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu zarządzania użytkownikami w panelu administratora		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-17, LAB-F-32, LAB-F-33. LAB-F-34		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-17	Priorytet:	M
Nazwa:	System powinien rozróżniać 3 role użytkowników: administrator, administrator grupy, użytkownik		
Opis:	Jako użytkownik systemu chcę aby posiadał on kontrolę dostępu w postaci ról użytkowników: administrator, administrator grupy oraz użytkownik		
Kryteria akceptacji:	System posiadający role użytkowników opisane w wymaganiu		
Dane wejściowe:	Brak		
Warunki początkowe:	System posiadający funkcjonalność autoryzacji użytkowników		
Warunki końcowe:	Możliwość wyboru roli pomiędzy: administrator, administrator grupy i użytkownik		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie pola „role” w bazie danych użytkowników		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:			

KARTA WYMAGANIA			
Identyfikator:	LAB-F-26	Priorytet:	M
Nazwa:	System musi posiadać panel dostępnych użytkowników		
Opis:	Jako użytkownik systemu chcę mieć możliwość identyfikacji użytkowników w systemie w przeznaczonym do tego panelu.		
Kryteria akceptacji:	System posiadający panel dostępnych użytkowników.		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie, przynajmniej 1 dodatkowy użytkownik w systemie.		
Warunki początkowe:	Użytkownik zalogowany do systemu w zakładce “Users”		
Warunki końcowe:	Użytkownik widzi innych użytkowników w liście wszystkich użytkowników.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu “Users”		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-27		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-27	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów użytkownika		
Opis:	Jako użytkownik systemu chcę mieć dostęp do szczegółów innego użytkownika żeby łatwo identyfikować jego przypisaną grupę lub informacje kontaktowe.		
Kryteria akceptacji:	Zaimplementowany panel szczegółów użytkownika		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie, przynajmniej 1 dodatkowy użytkownik w systemie.		
Warunki początkowe:	Użytkownik zalogowany do systemu w zakładce "Users" widzi innych użytkowników.		
Warunki końcowe:	Po kliknięciu w innego użytkownika z listy, użytkownik przenoszony jest do panelu szczegółów wybranego użytkownika.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu szczegółów użytkownika		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-26		

4.4.8 Zakładka 'Groups'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-18	Priorytet:	M
Nazwa:	System musi zezwalać na przypisanie oraz usunięcie użytkownika z istniejącej grupy		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chcę mieć możliwość przypisania oraz usunięcia użytkowników z istniejących grup co pozwoli efektywniej zarządzać zespołami w organizacji		
Kryteria akceptacji:	Funkcjonalność przypisywania użytkownika zaimplementowana w systemie		
Dane wejściowe:	System z funkcjonalną bazą danych		
Warunki początkowe:	System z przynajmniej 1 użytkownikiem w systemie		
Warunki końcowe:	Możliwość przypisania użytkownika do grupy		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie pola „group” w bazie danych użytkowników		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-19		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-19	Priorytet:	M
Nazwa:	System musi zezwalać na tworzenie nowych grup użytkowników oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik z uprawnieniami administratora chcę mieć możliwość tworzenia grup użytkowników oraz modyfikacji i usunięcia ich z poziomu systemu		
Kryteria akceptacji:	Zakładka „Groups” zaimplementowana w systemie		
Dane wejściowe:	użytkownik systemu z przypisaną rolą administratora		
Warunki początkowe:	użytkownik zalogowany do systemu		
Warunki końcowe:	Użytkownik widzi zakładkę „Groups” na pasku bocznym, po której kliknięciu przeniesiony zostaje na widok grup użytkowników, widoczny jest przycisk „Add Group”, po kliknięciu w który wyświetla się formularz tworzenia grupy		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie zakładki „Groups” na pasku bocznym, implementacja formularza dodania grupy		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-18		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-28	Priorytet:	M
Nazwa:	System musi posiadać panel dostępnych grup użytkowników		
Opis:	Jako użytkownik systemu chcę mieć dostęp do panelu z aktualnymi grupami użytkowników w celu łatwej identyfikacji utworzonych już w systemie grup.		
Kryteria akceptacji:	System z zaimplementowanym panelem “Groups”		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie, przynajmniej 1 grupa utworzona w systemie.		
Warunki początkowe:	Użytkownik zalogowany do systemu, na pasku bocznym widzi zakładkę “Groups”		
Warunki końcowe:	Po kliknięciu w zakładkę “Groups” użytkownik zostaje przeniesiony na panel grup użytkowników.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu grup użytkowników		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-29		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-29	Priorytet:	M
Nazwa:	System musi posiadać panel szczegółów grupy użytkowników		
Opis:	Jako użytkownik systemu chcę mieć dostęp do informacji o istniejących grupach użytkowników aby efektywnie wyszukiwać szczegółowe informacje o grupie oraz jej członkach.		
Kryteria akceptacji:	System z zaimplementowanym panelem szczegółów grupy użytkowników		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie, przynajmniej 1 grupa utworzona w systemie, zaimplementowana zakładka "Groups"		
Warunki początkowe:	Użytkownik zalogowany do systemu w zakładce "Groups"		
Warunki końcowe:	Po kliknięciu w wybraną grupę, użytkownik zostaje przeniesiony do panelu szczegółów grupy.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu szczegółów grupy		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-28		

4.4.9 Panel Administratora

KARTA WYMAGANIA			
Identyfikator:	LAB-F-20	Priorytet:	M
Nazwa:	System musi zezwalać na tworzenie nowych użytkowników oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik systemu z rolą administratora chcę mieć możliwość tworzenia nowych użytkowników oraz edycję i usunięcie istniejących.		
Kryteria akceptacji:	Funkcjonalność tworzenia, modyfikacji oraz usunięcia użytkowników i panel administratora dostępne w systemie		
Dane wejściowe:	Użytkownik z rolą administratora		
Warunki początkowe:	Użytkownik zalogowany do systemu widzi panel administratora na pasku bocznym		
Warunki końcowe:	Pod panelem administratora, użytkownik widzi zakładkę "users", po kliknięciu w którą przeniesiony jest do panelu, w którym ma możliwość utworzenia nowego użytkownika, modyfikacji lub usunięcia istniejącego		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Dodanie panelu administratora i implementacja logiki tworzenia nowego użytkownika oraz edycji i usunięcia istniejących.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-32	Priorytet:	M
Nazwa:	System musi posiadać panel administratora		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chcę mieć dostęp do panelu, z poziomu którego będę w stanie wykonywać akcje przeznaczone do administracji systemem.		
Kryteria akceptacji:	System z zaimplementowanym panelem administratora		
Dane wejściowe:	Użytkownik systemu z uprawnieniami administratora		
Warunki początkowe:	Użytkownik zalogowany do systemu, w pasku bocznym widzi zakładkę "Admin" poniżej której znajdują się moduły administracyjne		
Warunki końcowe:	Użytkownik jest w stanie przejść do każdego z modułów administracyjnych po kliknięciu w nie na pasku bocznym		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu administratora		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-09, LAB-F-17, LAB-F-33. LAB-F-34		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-33	Priorytet:	M
Nazwa:	Panel administratora musi posiadać panel zarządzania zespołami		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chce w panelu administratora posiadać dedykowany panel do zarządzania grupami użytkowników		
Kryteria akceptacji:	Panel administratora z zaimplementowanym panelem zarządzania zespołami		
Dane wejściowe:	Użytkownik systemu z uprawnieniami administratora		
Warunki początkowe:	Użytkownik zalogowany do systemu, w pasku bocznym widzi zakładkę "Admin" poniżej której znajdują się moduły administracyjne spośród których widoczny jest moduł "Groups"		
Warunki końcowe:	Po kliknięciu w moduł "Groups" użytkownik przenoszony jest do panelu zarządzania grupami użytkowników		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu zarządzania zespołami w panelu administratora		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-32		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-34	Priorytet:	M
Nazwa:	Panel administratora musi posiadać panel historii działań w wersji niesformatowanej		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chcę w panelu administratora posiadać dedykowany panel z historią zmian w postaci niesformatowanej by skutecznie identyfikować które komponenty systemu zostały zmienione.		
Kryteria akceptacji:	Panel administratora z zaimplementowanym panelem historii.		
Dane wejściowe:	Użytkownik systemu z uprawnieniami administratora		
Warunki początkowe:	Użytkownik zalogowany do systemu, w pasku bocznym widzi zakładkę "Admin" poniżej której znajdują się moduły administracyjne spośród których widoczny jest moduł "History"		
Warunki końcowe:	Po kliknięciu w moduł "Groups" użytkownik przenoszony jest do panelu historii działań w wersji niesformatowanej.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu historii działań w postaci niesformatowanej w panelu administratora		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-09, LAB-F-32. LAB-F-33, LAB-F-35		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-35	Priorytet:	M
Nazwa:	System musi zezwalać na reset hasła użytkownika		
Opis:	Jako użytkownik systemu z uprawnieniami administratora chcę mieć możliwość zresetowania hasła użytkownika aby umożliwić efektywne przywracanie dostępu do konta użytkownikowi		
Kryteria akceptacji:	Mechanizm resetu hasła użytkownika zaimplementowany w systemie		
Dane wejściowe:	Użytkownik systemu z przypisaną rolą administratora, dodatkowy użytkownik w systemie z dowolną rolą, przynajmniej 1 dodatkowy użytkownik w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, w pasku bocznym widzi zakładkę "Admin" poniżej której znajdują się moduły administracyjne spośród których widoczny jest moduł "Users" po kliknięciu w który przeniesiony jest do panelu zarządzania użytkownikami.		
Warunki końcowe:	Po kliknięciu w przycisk akcji na wybranym użytkowniku użytkownik widzi opcję resetu hasła		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie akcji resetu hasła w panelu zarządzania użytkownikami		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-09, LAB-F-32		

4.4.10 Import & Export

KARTA WYMAGANIA			
Identyfikator:	LAB-F-13	Priorytet:	C
Nazwa:	System powinien pozwalać użytkownikom na wybór między zastąpieniem i utworzeniem kopii danych importowanych z zewnętrznych źródeł		
Opis:	Jak użytkownik systemu chce mieć możliwość importu i eksportu danych z systemu do/z pliku		
Kryteria akceptacji:	System posiada funkcjonalny mechanizm importu i eksportu danych		
Dane wejściowe:	użytkownik systemu z rolą administratora		
Warunki początkowe:	użytkownik zalogowany do systemu, widzi zakładkę „Import & Export” na pasku bocznym		
Warunki końcowe:	Po kliknięciu w zakładkę „Import & Export” użytkownik przenoszony jest do formularza z możliwością importu i eksportu danych		
Sytuacje wyjątkowe:	Brak miejsca na dysku, plik w niewłaściwym formacie		
Szczegóły implementacji:	Dodanie zakładki „Import & Export” do paska bocznego, implementacja logiki eksportu i importu danych		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.4.11 Zakładka 'Documentation'

KARTA WYMAGANIA			
Identyfikator:	LAB-F-30	Priorytet:	M
Nazwa:	System musi posiadać panel dokumentacji technicznej		
Opis:	Jako użytkownik systemu chcę mieć dostęp do dokumentacji technicznej dostępnej w organizacji w przeznaczonym do tego panelu.		
Kryteria akceptacji:	System z zaimplementowanym panelem "Documentation"		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, na pasku bocznym widzi zakładkę "Documentation"		
Warunki końcowe:	Po kliknięciu w zakładkę "Documentation" użytkownik przenoszony jest na panel dokumentacji technicznej.		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Zaimplementowanie panelu dokumentacji technicznej		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-31		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-31	Priorytet:	M
Nazwa:	System musi posiadać panel podglądu, dodawania i edycji dokumentacji		
Opis:	Jako użytkownik systemu chcę mieć możliwość edycji oraz dodawania dokumentacji technicznej jak i możliwość podglądu utworzonych już dokumentów.		
Kryteria akceptacji:	Zakładka "Documentation" z zaimplementowanym panelem podglądu, dodawania i edycji dokumentu.		
Dane wejściowe:	Użytkownik z dowolną rolą w systemie.		
Warunki początkowe:	Użytkownik zalogowany do systemu w zakładce "Documentation"		
Warunki końcowe:	Po kliknięciu w wybrany dokument użytkownik widzi jego podgląd i przycisk do edycji oraz przycisk tworzenia nowego dokumentu.		
Sytuacje wyjątkowe:	Brak istniejącej dokumentacji w systemie		
Szczegóły implementacji:	Zaimplementowanie panelu modyfikacji i podglądu dokumentu. Zaimplementowanie przycisku tworzenia nowego dokumentu.		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-30		

4.4.12 Etykiety

KARTA WYMAGANIA			
Identyfikator:	LAB-F-44	Priorytet:	M
Nazwa:	System musi zezwalać na tworzenie nowych etykiet oraz edycję i usunięcie istniejących		
Opis:	Jako użytkownik systemu chcę mieć możliwość tworzenia etykiet aby być w stanie tworzyć oznaczenia elementów systemu		
Kryteria akceptacji:	Implementacja systemu etykiet w systemie		
Dane wejściowe:	Użytkownik systemu z dowolną przypisaną rolą		
Warunki początkowe:	Użytkownik zalogowany do systemu, na poziomie formularza dodawania nowej maszyny		
Warunki końcowe:	Użytkownik widzi pole etykiet, jest w stanie wpisać treść po czym utworzona zostaje etykieta		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja systemu etykiet w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-45		

KARTA WYMAGANIA			
Identyfikator:	LAB-F-45	Priorytet:	M
Nazwa:	System musi zezwalać na przypisanie 1 lub więcej etykiet do grupy użytkowników, maszyny, laboratorium oraz szafy rackowej		
Opis:	Jako użytkownik systemu chcę mieć możliwość przypisywania etykiet do elementów roboczych w systemie by efektywnie zarządzać zasobami		
Kryteria akceptacji:	Implementacja systemu przypisywania etykiet		
Dane wejściowe:	Użytkownik systemu z dowolną przypisaną rolą, przynajmniej 1 element roboczy przypisany do użytkownika lub jego grupy w systemie		
Warunki początkowe:	Użytkownik zalogowany do systemu, na poziomie szczegółów elementu		
Warunki końcowe:	Użytkownik widzi pole etykiet, jest w stanie wpisać treść po czym utworzona zostaje etykieta lub wybrać z listy istniejące etykiety		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja systemu przypisywania etykiet w systemie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-44		

4.5 Interfejs z otoczeniem

KARTA WYMAGANIA			
Identyfikator:	LAB-I-01	Priorytet:	M
Nazwa:	System musi zezwalać na import i eksport danych z plików csv		
Opis:	Jako użytkownik systemu chcę mieć możliwość importu i eksportu danych do/z plików plików csv		
Kryteria akceptacji:	Zaimplementowana logika importu z plików csv		
Dane wejściowe:	Użytkownik systemu z rolą administratora, zaimplementowana zakładka „Import & Export”		
Warunki początkowe:	Użytkownik zalogowany do systemu na ekranie formularza służącego do importu i eksportu danych		
Warunki końcowe:	Użytkownik jest w stanie importować oraz eksportować dane do pliku csv		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja logiki importu i eksportu do plików csv		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	LAB-F-13		

KARTA WYMAGANIA			
Identyfikator:	LAB-I-02	Priorytet:	C
Nazwa:	System powinien zezwalać na integrację z systemami PDU		
Opis:	Jako użytkownik systemu chcę mieć dostęp do systemów PDU z poziomu systemu		
Kryteria akceptacji:	Zaimplementowana obsługa systemów PDU		
Dane wejściowe:	użytkownik systemu z dowolną rolą, przynajmniej 1 urządzenie w systemie fizycznie podłączone do PDU		
Warunki początkowe:	użytkownik zalogowany do systemu, na ekranie szczegółów urządzenia widzi panel związany z systemem PDU		
Warunki końcowe:	użytkownik jest w stanie wykonywać akcję na systemie PDU korzystając z widoku szczegółów urządzenia		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja integracji z systemami PDU na poziomie szczegółów urządzenia		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-I-03	Priorytet:	M
Nazwa:	System musi zezwalać na import danych z agentów systemu Prometheus		
Opis:	Jako użytkownik systemu chcę widzieć dane z agentów systemu Prometheus na ekranie szczegółów urządzenia		
Kryteria akceptacji:	Zaimplementowana obsługa importu danych z agentów systemu Prometheus		
Dane wejściowe:	użytkownik systemu z dowolną rolą, przynajmniej 1 urządzenie posiadające zainstalowany system Prometheus		
Warunki początkowe:	użytkownik zalogowany do systemu, na poziomie widoku szczegółów urządzenia		
Warunki końcowe:	użytkownik widzi dane zaimportowane z systemu Prometheus		
Sytuacje wyjątkowe:	Brak		
Szczegóły implementacji:	Implementacja importu danych do systemu z agentów systemu Prometheus		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-I-04	Priorytet:	M
Nazwa:	System musi zezwalać na import danych z Ansible		
Opis:	Jako użytkownik chcę mieć dostęp do danych z Ansible		
Kryteria akceptacji:	Zaimplementowana logika importu danych z Ansible		
Dane wejściowe:	Przynajmniej 1 urządzenie dostępne w sieci lokalnej i w systemie Labbyn		
Warunki początkowe:	Wysłanie żądania używając API Ansible		
Warunki końcowe:	Żądanie przetworzone poprawnie, akcje zapisane w Playbook'u Ansible wykonane pomyślnie, dane zaimportowane do systemu		
Sytuacje wyjątkowe:	Zerwanie połączenia sieci lokalnej		
Szczegóły implementacji:	Zaimplementowanie obsługi przetwarzania żądań Ansible i importu danych do systemu		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.6 Wymagania pozafunkcjonalne

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-01	Priorytet:	S
Nazwa:	Przetwarzanie danych systemu		
Opis:	System powinien umożliwiać sprawne wprowadzanie dużych ilości masowych danych		
Kryteria akceptacji:	Odpowiednio zoptymalizowany system obsługi danych		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-02	Priorytet:	S
Nazwa:	Jednoczesne połączenia		
Opis:	System powinien umożliwiać jednoczesne połączenie wielu użytkowników.		
Kryteria akceptacji:	Implementacja systemu wielu połączeń do systemu jednocześnie		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-03	Priorytet:	M
Nazwa:	Autoryzacja użytkowników		
Opis:	System musi autoryzować wykonywane w nim akcje na podstawie roli/uprawnień użytkownika		
Kryteria akceptacji:	Zaimplementowany mechanizm autoryzacji użytkowników		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-04	Priorytet:	S
Nazwa:	Przenaszalność systemu		
Opis:	System powinien pozwolić na przenoszenie go między różnymi środowiskami		
Kryteria akceptacji:	Implementacja systemu w formie łatwej do przeniesienia		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-05	Priorytet:	S
Nazwa:	Utrzymanie i aktualizacja		
Opis:	System powinien być łatwy w utrzymaniu oraz aktualizacji		
Kryteria akceptacji:	Implementacja mechanizmu aktualizacji, uproszczenie i minimalizacja zależności systemu		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-06	Priorytet:	S
Nazwa:	Doświadczenia użytkownika		
Opis:	System powinien zapewnić standardy i dobre praktyki UX		
Kryteria akceptacji:	Implementacja systemu zgodnie z przyjętymi standardami UX		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-07	Priorytet:	M
Nazwa:	Domyślny użytkownik		
Opis:	System musi posiadać wbudowanego użytkownika serwis		
Kryteria akceptacji:	Implementacja autoryzacji pierwszego uruchomienia		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-NF-08	Priorytet:	M
Nazwa:	Przechowywanie haseł użytkowników		
Opis:	System musi przechowywać hasła użytkowników w formie zaszyfrowanej żeby zapewnić bezpieczeństwo danych		
Kryteria akceptacji:	Zaimplementowany mechanizm szyfrowania haseł po stronie systemu		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

4.7 Wymagania na środowisko docelowe

KARTA WYMAGANIA			
Identyfikator:	LAB-E-01	Priorytet:	M
Nazwa:	Wspierane przeglądarki		
Opis:	System musi być kompatybilny z przeglądarkami na silniku Chromium		
Kryteria akceptacji:	W pełni działający system dostępny w przeglądarkach na silniku Chromium		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-E-02	Priorytet:	S
Nazwa:	System operacyjny		
Opis:	System powinien działać i być kompatybilny z systemami Linux		
Kryteria akceptacji:	W pełni działający system Labbyn na systemach Linux		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

KARTA WYMAGANIA			
Identyfikator:	LAB-E-03	Priorytet:	M
Nazwa:	Środowisko domyślne		
Opis:	System musi działać w środowisku kontenerowym		
Kryteria akceptacji:	Implementacja systemu wspierającego środowisko kontenerowe		
Udziałowiec:	Zespół projektowy		
Wymagania powiązane:	Brak		

5 Architektura systemu Backend

5.1 Stack technologiczny

Warstwa Backendowa systemu, została zaimplementowana w języku Python w połączeniu z frameworkiem FastAPI.

Wybór takiego rozwiązania został podyktowany przede wszystkim: wysoką wydajnością oraz natywnym wsparciem dla operacji asynchronicznych, co jest kluczowe w systemach zarządzania dużymi zasobami.

Framework FastAPI zapewnia również automatyczną walidację danych wejściowych we współpracy z biblioteką Pydantic, co pozwala na minimalizację błędów w czasie działania aplikacji oraz na automatyczną generację dokumentacji API w standardzie OpenAPI.

Istotnym aspektem wyboru była także łatwa integracja z systemami bazodanowymi poprzez użycie systemu mapowania obiektowo-relacyjnego (ORM), dostarczonego przez bibliotekę SQL Alchemy 2.0. W połączeniu z bazą PostgreSQL co umożliwiło bezpieczne i transakcyjne zarządzanie zasobami zapewniając integralność danych.

Kolejnym istotnym czynnikiem była łatwość integracji z zewnętrznymi narzędziami i bibliotekami. Wykorzystany stack technologiczny pozwolił na sprawną implementację komunikacji z systemami monitoringu (Prometheus, Grafana) oraz narzędziami do automatyzacji konfiguracji (Ansible) służącymi do orkiestracji zasobów.

Architekturę Backendu wzbogacono o mechanizm Redis służący do zarządzania rozproszonymi blokadami zasobów, co jest kluczowe w przypadku środowisk wielodostępnych, gdzie jednoczesna modyfikacja zasobów przez wiele użytkowników może prowadzić do niespójności lub błędów race condition.

5.2 Architektura i wzorce projektowe

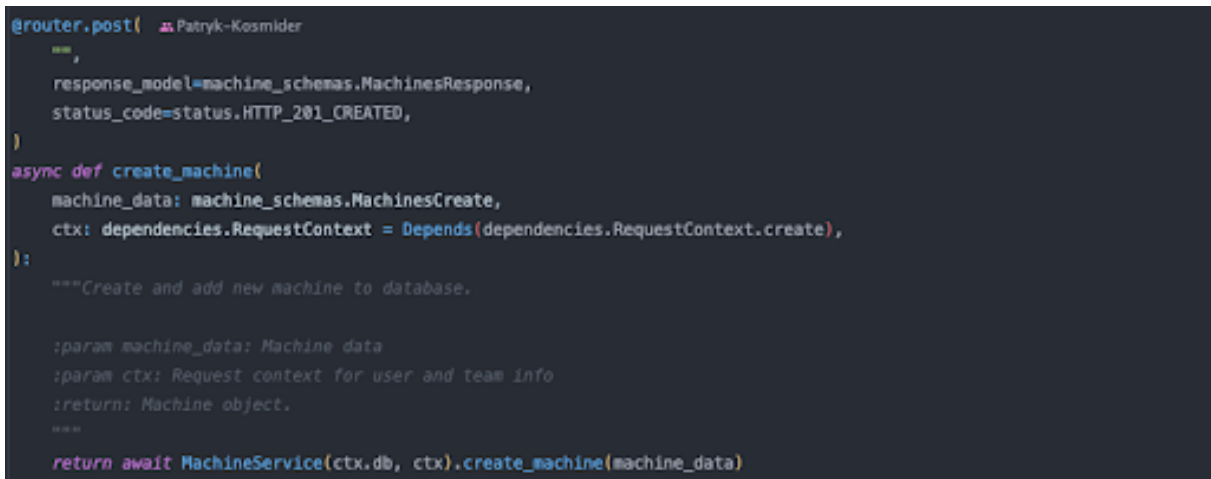
Warstwa backendowa została oparta na architekturze wielowarstwowej, której celem jest odseparowanie odpowiedzialności pomiędzy osobnymi komponentami. Takie rodzaj architektury ułatwia potencjalnie skalowanie systemu w przyszłości.

5.2.1 Warstwa interfejsu

Warstwa odpowiedzialna za obsługę protokołu HTTP, routing, walidację schematów, wstrzykiwanie zależności odpowiedzialnych za autoryzację i inicjalizację kontekstu użytkownika.

Zaprojektowana w architekturze REST, mapując następujące operacje odpowiednio:

- GET - pobieranie informacji o zasobach z bazy danych
- POST - wprowadzenie nowych obiektów do bazy danych
- PATCH/PUT - aktualizacja istniejących już obiektów w bazie danych
- DELETE - usuwanie istniejących obiektów w bazie danych

The image shows a code editor with a dark background. At the top left, there is a comment '@router.post()' followed by a small icon and the name 'Patryk-Kosmider'. The code defines a FastAPI endpoint. It starts with a docstring '"""', followed by 'response_model=machine_schemas.MachinesResponse,' and 'status_code=status.HTTP_201_CREATED,'. Then it closes the docstring with '"""'. Below this, it defines an asynchronous function 'create_machine' with a docstring '"""Create and add new machine to database.' and a return type annotation 'Machine object.'. The function takes 'machine_data: machine_schemas.MachinesCreate,' and 'ctx: dependencies.RequestContext = Depends(dependencies.RequestContext.create),' as arguments. It then calls 'return await MachineService(ctx.db, ctx).create_machine(machine_data)'.

```
@router.post()  Patryk-Kosmider
"""
response_model=machine_schemas.MachinesResponse,
status_code=status.HTTP_201_CREATED,
"""
)
async def create_machine(
    machine_data: machine_schemas.MachinesCreate,
    ctx: dependencies.RequestContext = Depends(dependencies.RequestContext.create),
):
    """Create and add new machine to database.

    :param machine_data: Machine data
    :param ctx: Request context for user and team info
    :return: Machine object.
    """
    return await MachineService(ctx.db, ctx).create_machine(machine_data)
```

Rysunek 5.1: Obsługa zapytań HTTP w module machines, odpowiadająca za stworzenie nowego obiektu

5.2.2 Warstwa serwisowa

Warstwa odpowiedzialna za przetwarzanie żądań dostarczonych przez użytkowników aplikacji, zgodnie z regułami biznesowymi które zostały w niej zdefiniowane. Łączy w sobie funkcjonalności wielu modułów, odpowiada za integralność i walidację biznesową, zarządza stanem i synchronizacją. Jest odizolowana od bazy danych, a także protokołu HTTP.

```

async def update_room(self, room_id: int, room_data): 1 usage  Patryk-Kosmider
    """Update room details and tags.

    :param room_id: Room ID to update.
    :param room_data: Update schema.
    :return: Updated room.
    :raises ConflictError: On name collision.
    """
    self.ctx.require_group_admin()
    async with redis_service.acquire_lock(f"room_lock:{room_id}"):
        room = await self.get_room_or_404(room_id)
        update_data = room_data.model_dump(exclude_unset=True)

        try:
            if "tag_ids" in update_data:
                tag_ids = update_data.pop("tag_ids")
                if tag_ids is not None:
                    tag_res = await self.db.execute(
                        sql.select(models.Tags).where(models.Tags.id.in_(tag_ids))
                    )
                    room.tags = tag_res.scalars().all()

            if "team_id" in update_data:
                await self.ctx.validate_team_access(update_data["team_id"])

            for k, v in update_data.items():
                setattr(room, k, v)

            await self.db.commit()
            await self.db.refresh(room, attribute_names=["team"])
            return room
        except IntegrityError:
            await self.db.rollback()
            raise exceptions.ConflictError(
                message=f"Conflict: Room name '{room.name}' already taken."
            )
        except Exception as e:
            await self.db.rollback()
            raise exceptions.ValidationError(
                f"Failed to update room '{room.name}'"
            ) from e

```

Rysunek 5.2: Logika biznesowa klasy RoomService, odpowiadająca za aktualizację wartości obiektu Room

5.2.3 Warstwa egzekutorów

Specjalna warstwa abstrakcji, dedykowana do współpracy z zewnętrznymi rozwiązaniami (Ansible, Prometheus). Pełni rolę interfejsu pomiędzy logiką biznesową a infrastrukturą zewnętrzną. W odróżnieniu od poprzednich warstw, egzekutorzy zarządzają pełnym cyklem życia wyżej wymienionych narzędzi. Dodatkowo są odpowiedzialni za hermetyzację logiki komunikacyjnej, zarządzanie stanem konfiguracyjnym, i prowadzenie autonomicznych procesów w tle.

```

class PrometheusExecutor: 5 usages  ⚡ Patryk-Kosmider
    """Handles direct communication with the Prometheus API and the targets JSON file."""

    URL = os.getenv("PROMETHEUS_URL")
    TARGETS_PATH = os.getenv("PROMETHEUS_TARGETS_PATH")

    STATUS_INTERVAL = int(os.getenv("HOST_STATUS_INTERVAL", 30))
    METRICS_INTERVAL = int(os.getenv("OTHER_METRICS_INTERVAL", 60))
    PUSH_INTERVAL = int(os.getenv("WEBSOCKET_PUSH_INTERVAL", 5))
    CACHE_STATUS_KEY = "prometheus_metrics_cache"
    CACHE_METRICS_KEY = "prometheus_other_metrics_cache"

    QUERIES = {
        "status": "up",
        "cpu_usage": "100 - (avg by (instance) (irate(node_cpu_seconds_total{node='idle'}[5m])) * 100)",
        "memory_usage": "(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes * 10",
        "disk_usage": "100 - (node_filesystem_avail_bytes{fstype!='tmpfs', mountpoint!='/boot'} * 100) / node_filesystem
    }

    _targets_lock = asyncio.Lock()

    @classmethod 1 usage  ⚡ Patryk-Kosmider
    async def _request(cls, url: str, params: dict, retries: int = 3) -> Dict[str, Any]:
        """Perform an asynchronous HTTP GET request with exponential backoff.

        :param url: The full destination URL (e.g., Prometheus query endpoint).
        :param params: A dictionary of query parameters.
        :param retries: Number of attempts before raising an error.
        :return: Parsed JSON response from the server.
        :raises exceptions.ExternalServiceError: If the service is unreachable or returns an error.
        """
        for _ in range(retries):
            try:
                async with httpx.AsyncClient(timeout=5.0) as client:
                    response = await client.get(url, params=params)
                    response.raise_for_status()
                    return response.json()
            except (httpx.HTTPError, asyncio.TimeoutError):
                await asyncio.sleep(0.5)
        raise exceptions.ExternalServiceError(
            service="Prometheus",
            detail=f"Failed to connect to Prometheus after {retries} attempts.",
        )

```

Rysunek 5.3: Początek klasy PrometheusExecutor, zarządzającej komunikacją z Prometheus

```

class AnsibleExecutor: 4 usages  Patryk-Kosmider
    """Class to handle the technical execution of Ansible playbooks.

    This executor manages the interaction with ansible_runner, handles dynamic
    inventory generation, and parses the resulting platform JSON reports
    stored on the file system.
    """

    REPORTS_DIR = "/code/ansible/platform_reports"
    PLAYBOOK_DIR = "/code/ansible"

    PLAYBOOK_MAP = {
        service_schemas.AnsiblePlaybook.create_user: f"{PLAYBOOK_DIR}/create_ansible_user.yaml",
        service_schemas.AnsiblePlaybook.scan_platform: f"{PLAYBOOK_DIR}/scan_platform.yaml",
        service_schemas.AnsiblePlaybook.deploy_agent: f"{PLAYBOOK_DIR}/deploy_agent.yaml",
        service_schemas.AnsiblePlaybook.delete_agent: f"{PLAYBOOK_DIR}/delete_agent.yaml",
        service_schemas.AnsiblePlaybook.delete_ansible: f"{PLAYBOOK_DIR}/delete_ansible.yaml",
    }

    @classmethod 2 usages  Patryk-Kosmider
    def parse_platform_report(cls, hostname: str) -> dict:
        """Reads and parses the JSON file generated by Ansible.

        This method extracts hardware and system information from a specific
        host's report file and maps it.

        :param hostname: The hostname of the machine whose report should be parsed.
        :return: A dictionary containing mapped system information (CPU, RAM, Disks, Network).
        """

        report_path = os.path.join(cls.REPORTS_DIR, f"{hostname}-platform-info.json")

        if not os.path.exists(report_path):
            raise exceptions.ObjectNotFoundError("Platform report", name=hostname)

        # Small delay to ensure the file system has finished writing the report
        time.sleep(1)
        try:
            with open(report_path, "r", encoding="utf-8") as f:
                data = json.load(f)
                root_key = list(data.keys())[0]
                info = data[root_key]

```

Rysunek 5.4: Początek klasy AnsibleExecutor, zarządzającej komunikacją i egzekucją funkcjonalności Ansible

5.2.4 Warstwa dostępu do danych

Warstwa działająca jako pomost pomiędzy modelem obiekowym, a relacyjną bazą danych. Zawiera ona wyłącznie surowe zapytania SQL, zbudowane za pomocą ORM. Operuje bezpośrednio na sesji bazy danych, a jej jedynym zadaniem jest wysłanie zapytania i odesłanie odpowiedzi do wyższych warstw.

```

class CategoryRepository: 4 usages  Patryk-Kosmider
    """Repository for handling Category database operations.

    Provides methods for CRUD operations on Categories model.
    """

    @staticmethod  Patryk-Kosmider
    async def get_all(db: AsyncSession) -> List[models.Categories]:
        """Fetch all categories from the database.

        :param db: Active asynchronous database session.
        :return: List of all Category model instances.
        """
        result = await db.execute(sql.select(models.Categories))
        return result.scalars().all()

    @staticmethod  Patryk-Kosmider
    async def get_by_id(db: AsyncSession, cat_id: int) -> models.Categories:
        """Fetch a specific category by its unique ID.

        :param db: Active asynchronous database session.
        :param cat_id: The primary key ID of the category.
        :return: Category model instance or None if not found.
        """
        result = await db.execute(
            sql.select(models.Categories).where(models.Categories.id == cat_id)
        )
        return result.scalar_one_or_none()

    @staticmethod  Patryk-Kosmider
    async def get_by_name(db: AsyncSession, name: str) -> models.Categories:
        """Fetch a specific category by its name.

        :param db: Active asynchronous database session.
        :param name: The unique name of the category.
        :return: Category model instance or None if not found.
        """
        result = await db.execute(
            sql.select(models.Categories).where(models.Categories.name == name)
        )
        return result.scalar_one_or_none()

    @staticmethod  Patryk-Kosmider
    async def delete(db: AsyncSession, category: models.Categories):
        """Delete a category instance from the database.

        :param db: Active asynchronous database session.
        :param category: The Category model instance to be deleted.
        """
        await db.delete(category)

```

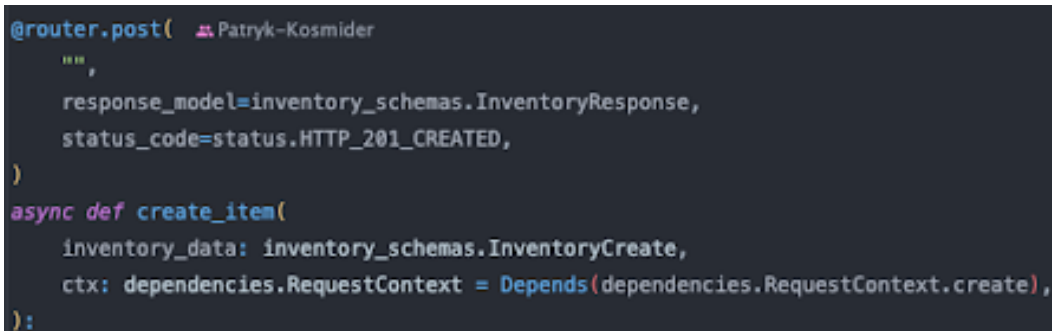
Rysunek 5.5: Repozytorium CategoryRepository, którego jedynym zadaniem jest komunikacja z bazą danych

5.3 Wzorce projektowe

W procesie tworzenia wykorzystano następujące wzorce projektowe, by zapewnić przejrzystość i modularność kodu.

5.3.1 Wstrzykiwanie zależności

Wzorzec ten wywodzi się z fundamentalnego działania frameworka FastAPI. Polega na przekazywaniu wymaganych komponentów do funkcji lub klas z zewnątrz zamiast tworzenia ich wewnątrz struktur. W systemie Labbyn wzorzec ten został wykorzystany do zarządzania sesjami połączeń do bazy danych oraz mechanizmem RequestContext (Szczegółowa implementacja mechanizmu kontekstu została opisana w podrozdziale 5.3.5), zawierającym tożsamość użytkownika oraz jego uprawnienia i role. Wzorzec jest stosowany w każdym istniejącym endpointzie aplikacji. Dzięki takiemu rozwiązaniu każdy z nich otrzymuje gotowy obiekt, unikając tworzenia połączeń wewnątrz swojego ciała, co jeszcze bardziej zwiększa jego niezależność od pozostałych modułów systemu.

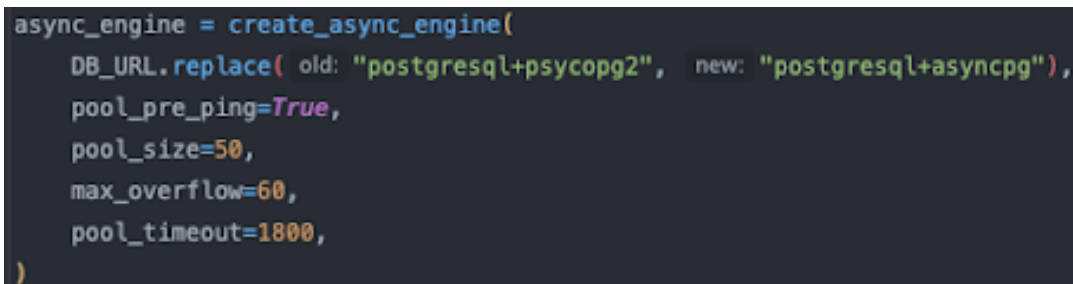


```
@router.post(
    """
    response_model=inventory_schemas.InventoryResponse,
    status_code=status.HTTP_201_CREATED,
)
async def create_item(
    inventory_data: inventory_schemas.InventoryCreate,
    ctx: dependencies.RequestContext = Depends(dependencies.RequestContext.create),
):
```

Rysunek 5.6: Endpoint służący do tworzenia nowego przedmiotu w inwentarzu ze wstrzyknięciem mechanizmu kontekstu

5.3.2 Singleton

Wzorzec gwarantujący, że dana klasa posiada tylko jedną instancję przez cały cykl życia aplikacji oraz globalny punkt dostępu. W systemie Labbyn wzorzec ten jest wykorzystywany do tworzenia silnika bazodanowego poprzez mechanizm cachowania modułów. Obiekt silnika jest globalny wewnątrz modułu bazy danych, przy pierwszym wywołaniu silnik bazodanowy jest inicjalizowany raz, a każda próba dostępu do niego, zawsze zwraca tą samą instancję.



```
async_engine = create_async_engine(
    DB_URL.replace( old: "postgresql+psycopg2", new: "postgresql+asyncpg"),
    pool_pre_ping=True,
    pool_size=50,
    max_overflow=60,
    pool_timeout=1800,
)
```

Rysunek 5.7: Funkcja create_async_engine z pliku database.py, tworzy globalnie dostępny silnik bazodanowy

Kolejnym przykładem zastosowanego wzorca jest menadżer pamięci podręcznej Redis. Klasa `RedisClientManager` przechowuje instancję klienta w atrybucie, a każda próba osiągnięcia go sprawdza najpierw czy instancja już istnieje oraz czy jest powiązana z aktualnie działającą pętlą zdarzeń zanim zostanie utworzona nowa.

```
class RedisClientManager: 1 usage  ⚠ Patryk-Kosmider
    """Singleton class to manage Redis Connection."""

    def __init__(self):  ⚠ Patryk-Kosmider
        """Initialize fields."""
        self.client = None
        self._loop = None

    async def get_client(self): 1 usage  ⚠ Patryk-Kosmider
        """Get a singleton Redis client instance."""
        try:
            current_loop = asyncio.get_running_loop()
        except RuntimeError:
            return await aioredis.from_url(REDIS_URL, decode_responses=True)
        if self.client is not None and self._loop is not current_loop:
            self.client = None
            self._loop = None

        if self.client is None:
            self.client = await aioredis.from_url(
                REDIS_URL, encoding="utf-8", decode_responses=True
            )
            self._loop = current_loop

        return self.client
```

Rysunek 5.8: Klasa `RedisClientManager`, tworzy globalnie dostępną pojedynczą instancję do komunikacji z pamięcią podręczną.

5.3.3 Data Transfer Object

Wzorzec służący do przesyłania danych między poszczególnymi elementami aplikacji, w sposób który nie ujawnia szczegółów implementacji modeli bazodanowych. W systemie Labbyn wzorzec ten jest wprowadzony dzięki wykorzystaniu biblioteki `Pydantic`. Schematy DTO definiują to, co dokładnie widzi użytkownik pozwalając wykluczać poszczególne wrażliwe elementy bazy danych, takich jak np. hasła lub ograniczać potrzebne pola, jeśli operacja nie wymaga ich wszystkich.

```

class User(SQLAlchemyBaseUserTable[int], Base):  # Patryk-Kosmider
    """User model representing users in the system."""

    __tablename__ = "user"

    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    surname = Column(String(80), nullable=False)
    login = Column(String(30), nullable=False, unique=True)
    email = Column(String(100), unique=True, index=True, nullable=False)
    avatar_path = Column(
        String(255), nullable=True, default="/static/avatars/default.png"
    )  # dummy path
    hashed_password = Column(String(255), nullable=False)

    # FastAPI Users fields
    is_active = Column(Boolean, default=True, nullable=False)
    is_superuser = Column(Boolean, default=False, nullable=False)
    is_verified = Column(Boolean, default=False, nullable=False)

    user_type = Column(
        Enum(UserType, name="user_type_enum", create_type=True),
        nullable=False,
        default=UserType.USER,
    )
    force_password_change = Column(Boolean, nullable=False, default=False)
    __table_args__ = {"schema": None}

    version_id = Column(Integer, nullable=False, default=1)

    __mapper_args__ = {"version_id_col": version_id}

    @property  # Patryk-Kosmider
    def team_name(self) -> str:
        if "teams" not in self.__dict__:
            return None
        return self.team.name if self.team else None

    teams = relationship("UsersTeams", back_populates="user")
    rentals = relationship("Rentals", back_populates="user")
    history = relationship("History", back_populates="user")

```

Rysunek 5.9: Klasa modelująca tabelę User z bazy danych do kodu

```

class UserBase(BaseModel): 2 usages  Patryk-Kosmider
    """Base model for Users containing shared public attributes.

    NOTE: Does NOT contain password.
    """

    name: str = Field(..., max_length=50, description="User's first name")
    surname: str = Field(..., max_length=80, description="User's last name")
    login: str = Field(..., max_length=30, description="Unique login username")
    email: Optional[EmailStr] = Field(None, description="User's email address")
    user_type: base_schemas.UserTypeEnum = Field(
        ..., max_length=50, description="User's role in the system"
    )

```

Rysunek 5.10: Schemat DTO, służący jako podstawowy schemat dla operacji związanych z użytkownikiem, powstały poprzez wykluczenie niepotrzebnych pól.

5.3.4 Observer

Wzorzec który służy do informowania obiektów o różnych, dowolnych zdarzeniach zachodzących w obiektach, które są przez niego obserwowane. W systemie Labbyn wzorzec ten został wykorzystany w systemie audytu. Obiektem obserwowanym jest sesja bazy danych, podczas której “obserwatorzy” rejestrują zmiany wprowadzone do określonych tabel (Machines, Inventory, Rooms, Users, Categories), i zapisują je automatycznie w tabeli History, przeznaczonej na składowanie logów (Szczegółowa implementacja mechanizmu logowania została opisana w podrozdziale 5.3.6). Dzięki takiemu zastosowaniu system Labbyn posiada automatyczny system logowania z możliwością łatwej manipulacji, np. poszerzeniu tabel które nas interesują.

```

@event.listens_for(orm.Session, "after_flush")  # Patryk-Kosmider
def receive_after_flush(session: orm.Session, flush_context: orm.UOWTransaction):
    """SQLAlchemy session listener triggered after data is flushed to database.

    This function is primarily used to handle logging for CREATE actions,
    as the primary key ('entity_id') of the newly created objects is now available.

    :param session: Current SQLAlchemy Session object
    :param flush_context: Unit of work transaction context
    :return: None
    """
    user_id = session.info.get("user_id", 1)

    objects = session.info.pop("objects_to_create_history", [])
    if not objects:
        return

    for obj in objects:
        if obj.id is None:
            continue

        entity_type = identify_entity_type(obj)
        if entity_type:
            session.add(
                models.History(
                    entity_type=entity_type,
                    action=models.ActionType.CREATE,
                    entity_id=obj.id,
                    user_id=user_id,
                    after_state=get_entity_state(obj),
                )
            )

```

Rysunek 5.11: Obserwator rejestrujący zmiany dotyczące tworzenia nowych obiektów

5.4 Konfiguracja i środowisko

Konfiguracja systemu Labbyn jest podzielona na statyczne zarządzanie zmiennymi środowiskowymi obsługiwanych za pomocą plików rozszerzeniem `.env`. Klucze dostępowe, hasła do bazy danych lub porty i adresy usług są przechowywane w plikach środowiskowych, np. `api.env`. Pozwala to na scentralizowane konfigurowanie środowiska, jeśli aplikacja ma zostać uruchomiona na innych portach niż domyślne, wystarczy nadpisać konkretną wartość.

```
PROMETHEUS_URL=http://monitoring:9090
REDIS_URL=redis://redis:6379/0
COLLECT_TIMEOUT=120
HOST_STATUS_INTERVAL=5
OTHER_METRICS_INTERVAL=5
WEBSOCKET_PUSH_INTERVAL=5
PROMETHEUS_TARGETS_PATH=/app/monitoring/prometheus-config/targets.json

ANSIBLE_HOST_KEY_CHECKING=False

DB_USER=admin
DB_PASSWORD=admin
DB_NAME=inventory

DB_HOST=db
DB_PORT=5432
ENV=production

DATABASE_URL="postgresql+psycopg2://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}"
|
```

Rysunek 5.12: Przykładowy plik `api.env`, konfigurujący parametry startowe aplikacji i zmienne globalne

W aplikacji również znajdują się mechanizm zarządzania cyklem życia, który również odpowiada za przygotowanie jej infrastruktury do pracy oraz zarządza zasobami, uruchamiany na samym starcie.

Podczas startu, system automatycznie tworzy potrzebną podstawową strukturę, auto inicjalizacja domyślnego admina i auto inicjalizacja pierwszego laboratorium. Serwer wykrykuje również istnienie katalogów na zasoby statyczne takie jak folder z avatarą użytkownika. Mechanizm również inicjuje procesy działające w tle służące do monitorowania infrastruktury poprzez moduł Prometheus, a jego prostota pozwala na rozbudowę o kolejne dodatkowe procesy.

```

@asynccontextmanager 1 usage  Patryk-Kosmider
async def lifespan(fast_api_app: FastAPI):
    """Application lifespan context manager.

    Starts background tasks for fetching Prometheus metrics and initializes DB.
    """

    db = AsyncSessionLocal()
    try:
        await database_service.init_super_user(db)
        await database_service.init_virtual_lab(db)
        await database_service.init_document(db)
    finally:
        await db.close()

    status_task = asyncio.create_task(PrometheusExecutor.status_worker())
    metrics_task = asyncio.create_task(PrometheusExecutor.metrics_worker())

    try:
        yield
    finally:
        status_task.cancel()
        metrics_task.cancel()
        await asyncio.gather(status_task, metrics_task, return_exceptions=True)

```

Rysunek 5.13: Menedżer kontekstu inicjalizujący życie aplikacji

Kolejnym etapem środowiska jest konfiguracja CORS, aby ograniczyć dostęp do API wyłącznie do zaufanych źródeł, by zapobiec nieautoryzowanym żądaniom, oraz klasa StaticFiles, służąca do wystawienia katalogu pod konkretnym adresem (/static/avatars), pod którym przechowywane będą grafiki służące jako awatary użytkowników.

```
app.mount(  
    "/static/avatars",  
    StaticFiles(directory=avatar_path),  
    name="avatars",  
)  
  
origins = [  
    "http://localhost:3000",  
    "http://127.0.0.1:3000",  
)  
  
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=origins,  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

Rysunek 5.14: Kod inicjalizujący CORS oraz montowanie folderu dla plików statycznych

Ostatnim etapem jest podłączenie routerów. Wszystkie logiczne moduły takie jak machine, rack lub tags, posiadają własne instancje klasy Router. Wszystkie wymienione zostają podłączone do głównego cyklu życia aplikacji, co pozwala też na bardzo proste podłączenie nowych endpointów lub rezygnację z konkretnych funkcjonalności.

```

app.include_router(
    auth_config.fastapi_users.get_auth_router(auth_config.auth_backend),
    prefix="/auth",
    tags=["auth"],
)

app.include_router(
    auth_config.fastapi_users.get_users_router(
        user_schemas.FastApiUserRead, user_schemas.FastApiUserUpdate
    ),
    prefix="/users",
    tags=["users"],
)

all_app_routers = [
    routers.auth,
    routers.user,
    routers.team,
    routers.room,
    routers.maps,
    routers.rack,
    routers.shelf,
    routers.machine,
    routers.cpus,
    routers.disks,
    routers.inventory,
    routers.category,
    routers.rental,
    routers.metadata,
    routers.tags,
    routers.documentation,
    routers.ansible,
    routers.prometheus,
    routers.dashboard,
    routers.history,
    routers.history_sub,
    routers.search,
    routers.import_csv,
    routers.export_csv,
]

for r in all_app_routers:
    <!--
    app.include_router(r)

```

Rysunek 5.15: Kod podłączający poszczególne mniejsze routery, by stworzyć pełen interfejs API

Całość znajduje się w pliku głównym `main.py`, co pozwala również na szybkie i proste dostosowanie aplikacji pod konkretne działanie, przez wprowadzenie minimalnych zmian.

5.5 Modele danych

W systemie Labbyn warstwa danych została zrealizowana za pomocą mapowania obiektowo-relacyjnego, z wykorzystaniem biblioteki SQLAlchemy. Pozwala to na definiowanie encji jako klas, zarządzanie typami oraz relacjami pomiędzy tabelami.

Wszystkie klasy modeli dziedziczą po wspólnej klasie bazowej, dzięki temu system automatycznie rejestruje tabele i ich powiązania. Każda kolumna w tabelach jest ściśle

opisana poprzez parametry określające typ, relacje, i dodatkowe punkty takie jak np. możliwość pozostania pustą lub reguły usunięcia.

Do określenia relacji zastosowany został mechanizm relationship, który pozwala na dociąganie powiązanych ze sobą obiektów w całości, np. wyciągnięcie maszyn przypisanych do konkretnej szafy rackowej, bez potrzeby budowania skomplikowanych zapytań.

Dodatkowo część modeli jest wyposażona w atrybuty obliczalne, za pomocą dekoratora @property, który pozwala na tworzenie wirtualnych pól. Oznacza to że model nie musi posiadać konkretnego pola w swojej definicji, a my możemy je dynamicznie wyciągnąć jeśli wynika ono z jego relacji z innym modelem.

Podstawowe encje, takie jak Machines, Rack lub Inventory, tworzące największą podstawę systemu, zostały wzbogacone o pole version_id, służące do wprowadzenia blokady optymistycznej.

```

class Machines(Base):  # Patryk-Kosmider +1
    """Machines model representing machines in the system."""

    __tablename__ = "machines"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    localization_id = Column(Integer, ForeignKey("rooms.id"), nullable=False)
    mac_address = Column(String(17), nullable=True)
    ip_address = Column(String(15), nullable=True)
    pdu_port = Column(Integer, nullable=True)
    team_id = Column(Integer, ForeignKey("teams.id"), nullable=True)
    os = Column(String(30), nullable=True)
    serial_number = Column(String(50), nullable=True)
    note = Column(String(500), nullable=True)
    added_on = Column(DateTime, nullable=False, default=datetime.now)
    ram = Column(String(100), nullable=True)
    metadata_id = Column(Integer, ForeignKey("metadata.id"), nullable=False)
    shelf_id = Column(Integer, ForeignKey("shelves.id"), nullable=True)
    version_id = Column(Integer, nullable=False, default=1)

    __mapper_args__ = {"version_id_col": version_id}

    __table_args__ = (
        UniqueConstraint("name", "localization_id", name="_machine_room_uc"),
    )

    @property  # Patryk-Kosmider
    def team_name(self):
        if "team" not in self.__dict__:
            return None
        return self.team.name if self.team else None

    @property  # Patryk-Kosmider
    def room_name(self):
        if "room" not in self.__dict__:
            return None
        return self.room.name if self.room else None

    room = relationship("Rooms", back_populates="machines")
    team = relationship("Teams", back_populates="machines")
    machine_metadata = relationship("Metadata", back_populates="machines")
    inventory = relationship("Inventory", back_populates="machine")
    shelf = relationship("Shelf", back_populates="machines", cascade="save-update, merge")
    tags = relationship("Tags", secondary="tags_machines", back_populates="machines")
    cpus = relationship("CPUs", back_populates="machine", cascade="all, delete-orphan")
    disks = relationship(
        "Disks", back_populates="machine", cascade="all, delete-orphan"
    )

```

Rysunek 5.16: Model bazodanowy reprezentujący obiekt Machine wraz z użyciem blokady optymistycznej i atrybutów obliczalnych

W celu wprowadzania zmian w strukturze bazy danych, zastosowano narzędzie Alembic. Każda zmiana w modelach jest rejestrowana jako osobny skrypt migracyjny. Dzięki takiemu rozwiązaniu jesteśmy w stanie odtworzyć strukturę bazy danych na dowolnym etapie.

Narzędzie Alembic potrafi również automatycznie wykrywać różnice pomiędzy modelami w kodzie a bazą danych, generując gotowe skrypty SQL i wprowadzać je na bieżąco,

by stan modeli zgadzał się z faktycznym stanem znajdującym się w bazie danych.

Dostarczany przez nie mechanizm migracji, pozwala na łatwe cofnięcie zmian w przypadku wykrycia błędów, by nie powodować przestoju w działaniu systemu. Każda zmiana struktury modeli, jest automatycznie rejestrowana jako rewizja, nowsza od poprzedniej, tworząc również łańcuch historii zmian.

```
"""Auto init on start

Revision ID: 863c4fd0e107
Revises: 5f68cfb2c196
Create Date: 2026-04-25 10:34:10.888061

"""

import ...

# revision identifiers, used by Alembic.
revision: str = '863c4fd0e107'
down_revision: Union[str, Sequence[str], None] = '5f68cfb2c196'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    """Upgrade schema."""
    # ## commands auto generated by Alembic - please adjust! ##
    op.create_table('categories',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(length=50), nullable=False),
        sa.Column('version_id', sa.Integer(), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('name')
    )
    op.create_table('documentation',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('title', sa.String(length=50), nullable=False),
        sa.Column('author', sa.String(length=50), nullable=False),
        sa.Column('content', sa.String(length=5000), nullable=True),
        sa.Column('added_on', sa.DateTime(), nullable=False),
        sa.Column('modified_on', sa.DateTime(), nullable=True),
        sa.Column('version_id', sa.Integer(), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('title')
    )
```

Rysunek 5.17: Przykładowy wygenerowany automatycznie w przypadku wprowadzenia zmian do modeli, skrypt migracyjny.

5.6 Autentykacja

W warstwie autentykacji, skorzystano z rozwiązania dostarczonego przez bibliotekę FastAPI Users, która została zintegrowana ze strategią bazodanową.

Zastosowano mechanizm BearerTransport, po poprawnym zalogowaniu się użytkownik otrzymuje unikatowy token przesyłany w nagłówku HTTP. Token ten zostaje zapisany do rekordu w dedykowanej tabeli access_tokens. W przeciwieństwie do standardowego rozwiązania za pomocą tokenów JWT, ta strategia pozwala na natychmiastowe przerwa-

nie sesji w przypadku wykrycia nieprawidłowości, lub usunięcia konta użytkownika, co pozwala na jeszcze większe podniesienie standardów bezpieczeństwa.

```
bearer_transport = BearerTransport(tokenUrl="auth/login")

def get_database_strategy( 2 usages (1 dynamic) 🐞 Patryk-Kosmider
    access_token_db: SQLAlchemyAccessTokenDatabase = Depends(get_access_token_db),
) -> DatabaseStrategy:
    """Create a database strategy for authentication.

    :param access_token_db: access token database dependency
    :return: DatabaseStrategy instance
    """

    return DatabaseStrategy(access_token_db, lifetime_seconds=None)

auth_backend = AuthenticationBackend(
    name="database-tokens",
    transport=bearer_transport,
    get_strategy=get_database_strategy,
)

fastapi_users = FastAPIUsers(models.User, int)(manager.get_user_manager, [auth_backend])

current_active_user = fastapi_users.current_user(active=True)
```

Rysunek 5.18: Konfiguracja autentykacji i strategii przechowywania tokenów

By zarządzać procesem logowania, skorzystano z gotowego mechanizmu dostarczonego przez FastAPI Users, który został poszerzony ręcznie, specjalnie w celu dostosowania go do systemu Labbyn.

Standardowe mechanizmy logowanie zostały nadpisane by umożliwić identyfikację użytkowników poprzez unikalny login, co jest bardziej praktyczne w rozwiązaniach wewnętrznych. Dzięki temu system w pierwszej kolejności weryfikuje tożsamość na podstawie loginu, następnie korzysta z wbudowanej funkcji do procesu weryfikacji hasła, które jest zabezpieczone za pomocą BCrypt. Każde hasło przed zapisem do bazy jest poddawane procesowi hashowania.

```

class UserManager(IntegerIDMixin, BaseUserManager[models.User, int]): 1 usage  Patryk-Kosmider
    """Custom user manager for handling user authentication and management."""

    reset_password_token_secret = AUTH_SECRET
    verification_token_secret = AUTH_SECRET

    async def get_by_login(self, login: str): 1 usage  Patryk-Kosmider
        """Retrieve a user by their login.

        :param login: The login of the user to retrieve.
        :return: The User instance if found.
        """
        query = sql.select(models.User).where(models.User.login == login)
        result = await self.user_db.session.execute(query)
        user = result.scalar_one_or_none()
        if user is None:
            raise exceptions.UserNotExists(f"User {login} not found.")
        return user

    async def authenticate(self, credentials):  Patryk-Kosmider
        """Authenticate a user with given credentials.

        :param credentials: The credentials to authenticate.
        :return: The authenticated User instance or None if authentication fails.
        """
        try:
            user = await self.get_by_login(credentials.username)
        except exceptions.UserNotExists:
            self.password_helper.hash(credentials.password)
            return None

        verified, _ = self.password_helper.verify_and_update(
            credentials.password, user.hashed_password
        )
        if not verified or not user.is_active:
            return None

        return user

    async def get_user_manager(user_db=Depends(get_user_db)): 2 usages (2 dynamic)  Patryk-Kosmider
        """Dependency generator that yields a UserManager instance.

        :param user_db: Database dependency instance.
        :return: UserManager instance.
        """
        yield UserManager(user_db)

```

Rysunek 5.19: Klasa UserManager, poszerzona o możliwość logowania się za pomocą loginu

W celu zapewnienia maksymalnej kontroli nad dostępem do infrastruktury, wdrożono własny system zarządzania hasłami. W przypadku inicjalizacji konta, system generuje jednorazowe hasło startowe, który jest przekazywane przez administratora użytkownikowi. Przy pierwszym logowaniu system zmusza użytkownika do zdefiniowania własnego hasła. W przypadku resetu hasła, administrator ustawia hasło tymczasowe i aktywuje flagę resetu. Następnie analogicznie przekazuje nowo wygenerowane jednorazowe hasło użytkownikowi, który jest zmuszony po zalogowaniu zmienić je na własne.

```

async def setup_first_password(self, data): 1 usage  Patryk-Kosmider
    """Setup first password.

    After creating user with automatically assigned password, the user needs
    to change it at his first login.

    :param data: Password change request data
    :return: Success or error message
    """

    if not self.user.force_password_change:
        raise exceptions.ValidationError(
            "Password change is not required for this account."
        )

    db_user = await self.repo.get_user_by_id(self.db, self.user.id)

    if not db_user:
        raise exceptions.ObjectNotFoundError("User", self.user.login)

    db_user.hashed_password = security.hash_password(data.new_password)
    db_user.force_password_change = False
    self.db.add(db_user)
    await self.db.commit()

    return {"message": "Password has been set successfully."}

async def force_password_reset(self, user_id: int): 1 usage  Patryk-Kosmider
    """Setup password change flag.

    Set flag to force password reset to redirect user to
    password reset subpage.

    :param user_id: User ID
    :return: Success or error message
    """

    db_user = await self.repo.get_user_by_id(self.db, user_id)

    if not db_user:
        raise exceptions.ObjectNotFoundError("User")

    temp_password = security.generate_starting_password()

    db_user.hashed_password = security.hash_password(temp_password)
    db_user.force_password_change = True

    self.db.info["user_id"] = self.user.id

    self.db.add(db_user)
    await self.db.commit()

    return {
        "message": f"Password reset forced for user {db_user.login}",
        "login": db_user.login,
        "password": temp_password,
    }

```

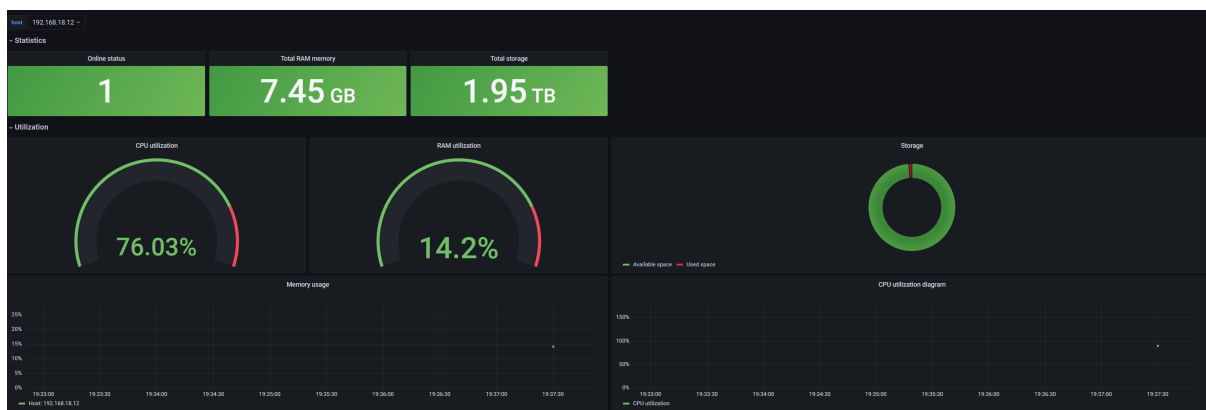
Rysunek 5.20: Klasa AuthService, serwis posiadający metody do resetowania hasła i obsługi pierwszego zalogowania

6 Przebieg sprintów

6.1 Sprint 0

Czas trwania sprintu: 29.10.2025 - 12.11.2025 (2 tygodnie)

Sprint ten był inicjalizacją prac nad projektem. Pierwszym zadaniem było zrobienie listy kluczowych funkcjonalności jakie system powinien posiadać. Aby uzupełnić potencjalne braki zespół stworzył ankietę, która została przekazana potencjalnym użytkownikom systemu. Po upływie 1 tygodnia od rozesłania ankiety, zbieranie odpowiedzi zostało zakończone. W tym czasie zespół stworzył POC systemu do monitorowania statystyk urządzeń tworząc pliki konfiguracyjne kompatybilne z systemem monitorowania Prometheus.

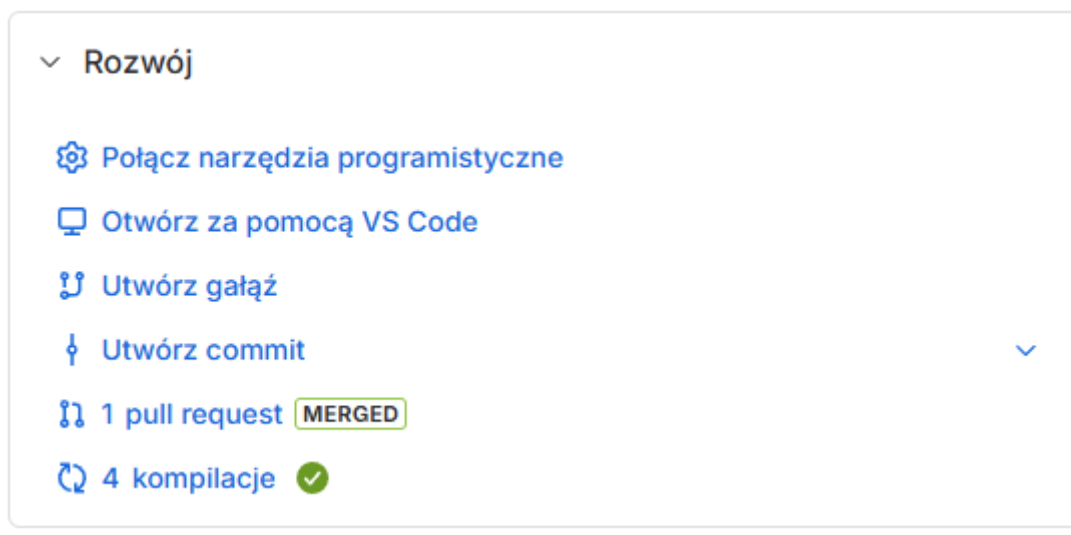


Rysunek 6.1: Zdjęcie przedstawiające panel monitorowania urządzeń systemu Prometheus

repozytoria dla kodu źródłowego: aplikacji oraz tego dokumentu. Repozytoria te zostały skonfigurowane w odpowiedni sposób, by zapobiec wprowadzaniu zmian bezpośrednio do głównej gałęzi, a każda zmiana musiała być poprzedzona Pull Request'em, którego zaakceptować musi:

- w przypadku kodu źródłowego aplikacji - przynajmniej 1 członek zespołu projektowego,
- w przypadku kodu źródłowego dokumentu - cały zespół projektowy.

Repozytorium zostało połączone z serwisem Jira w celu sprawniejszego wyszukiwania zmian w kodzie na podstawie zgłoszeń.



Rysunek 6.2: Zdjęcie przedstawiające integrację seriwsów GitHub oraz Jira

Po uzyskaniu odpowiedzi w wyznaczonym czasie, utworzone zostało sprawozdanie z ankiety opisane szczegółowo w punkcie 3.3 . Na jego podstawie wyspecyfikowane zostały wymagania na system, które zostały sklasyfikowane według dwóch wymiarów:

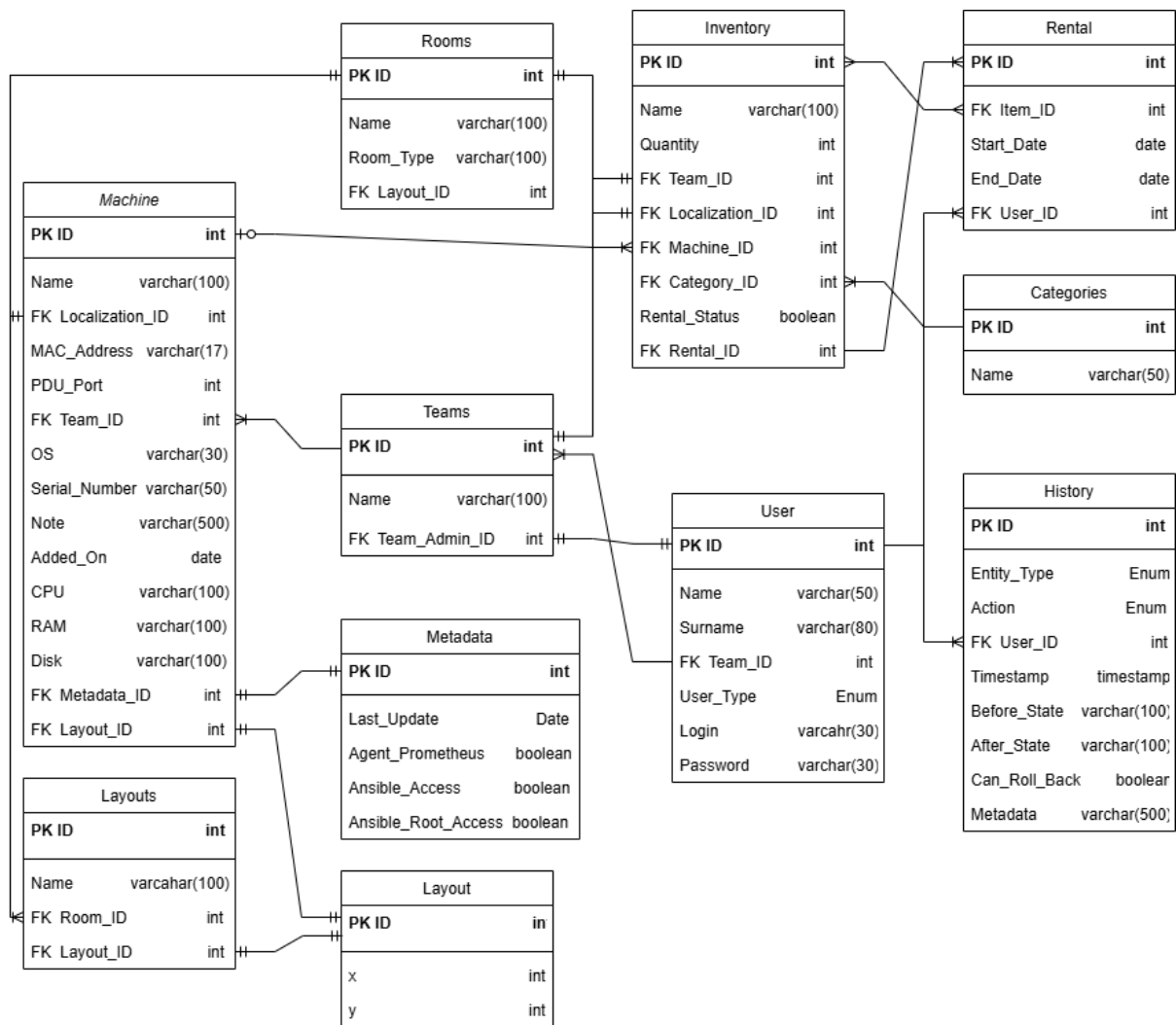
- typu wymagania (m.in. dziedzinowe, funkcjonalne, niefunkcjonalne),
- przynależności do poszczególnych modułów/obszarów funkcjonalnych (np. Panel Urządzenia, Search Bar).

Dzięki temu wytworzone zostały dokumenty takie jak: Karta Projektu, Dokument Założeń Wstępnych czy Specyfikacja Wymagań Systemowych.

6.2 Sprint 1

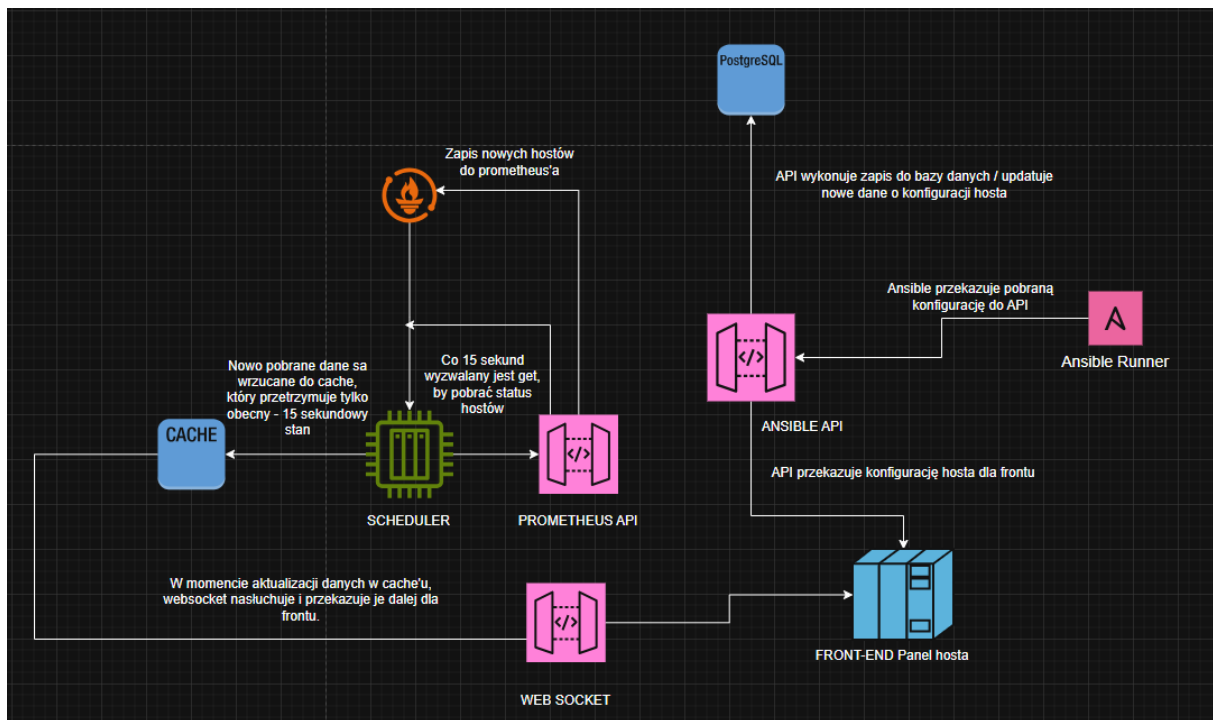
Czas trwania Sprintu: 12.11.2025 - 26.11.2025 (2 tygodnie)

Początkiem sprintu utworzony został wstępny model fizyczny bazy danych aplikacji, który wygląda następująco:



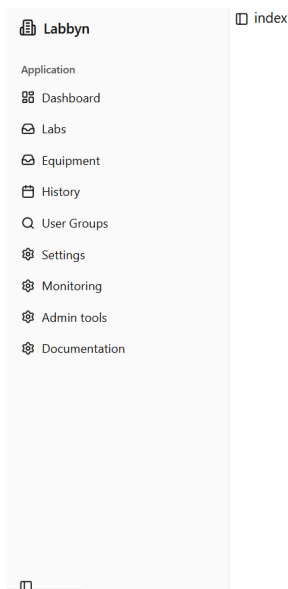
Rysunek 6.3: Wstępny projekt modelu fizycznego bazy danych

Model ten posłużył do wstępnej konfiguracji środowiska produkcyjnego oraz umożliwił rozpoczęcie prac nad logiką aplikacji. Stworzony został POC architektury systemu telemetrii urządzeń w postaci grafu:



Rysunek 6.4: POC systemu telemetry urządzeń

Powstała również wersja demonstracyjna aplikacji w celu przedstawienia zespołowi proponowanej wizji artystycznej projektu:



Rysunek 6.5: Wstępna wersja interfejsu użytkownika

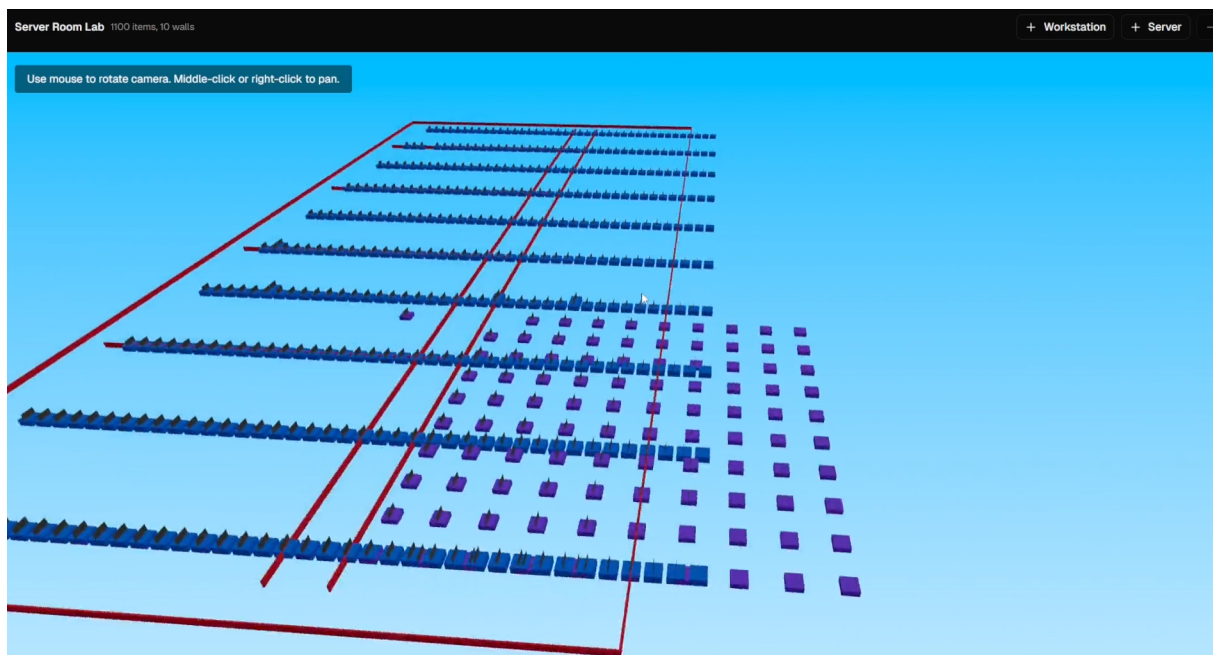
Utworzono podstawowy proces CI/CD wspierający budowanie i testowanie aplikacji, w ramach którego dodano linter jako element workflow CI w GitHubie. Przygotowane zostało również środowisko dewepolerskie umożliwiające obserwowanie zmian w oprogramowaniu w czasie rzeczywistym co zmniejszyło ryzyko powstawania błędów i pozwoliło

drastycznie zmniejszyć czas sprawdzania kompatybilności zmian - programista nie był zmuszony budować aplikacji od nowa co było czasochłonne.

6.3 Sprint 2

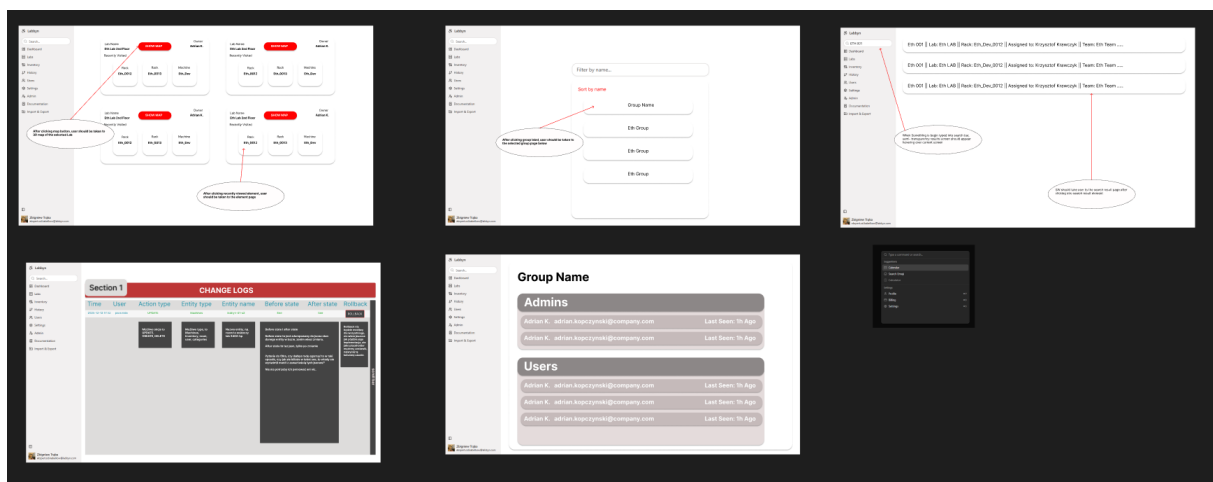
Czas trwania Sprintu: 26.11.2025 - 10.12.2025 (2 tygodnie)

Podczas tego sprintu powstał pierwszy prototyp mapy przestrzennej:



Rysunek 6.6: Prototyp mapy przestrzennej laboratorium

Ze względu na brak jednoznacznie zdefiniowanego konceptu wizualnego systemu, przygotowano mockupy w celu ujednolicenia kierunku projektu.



Rysunek 6.7: Zdjęcie przedstawiające część mockupów stworzonych dla systemu Labbyn

Zaimplementowano warstwę dostępu do bazy danych z wykorzystaniem SQL Alchemy.

6.4 Sprint 3

Czas trwania Sprintu: 10.12.2025 - 07.01.2026 (4 tygodnie)

6.5 Sprint 4

Czas trwania Sprintu: 07.01.2026 - 28.01.2026 (3 tygodnie)

6.6 Sprint 5

Czas trwania Sprintu: 28.01.2026 - 11.02.2026 (2 tygodnie)

6.7 Sprint 6

Czas trwania Sprintu: 11.02.2026 - 25.02.2026 (2 tygodnie)

6.8 Sprint 7

Czas trwania Sprintu: 25.02.2026 - 04.03.2026 (1 tygodnie)

6.9 Sprint 8

Czas trwania Sprintu: 04.03.2026 - 18.03.2026 (2 tydzień)

6.10 Sprint 9

Czas trwania Sprintu: 18.03.2026 - 01.04.2026 (2 tygodnie)

6.11 Sprint 10

Czas trwania Sprintu: 01.04.2026 - 15.04.2026 (2 tygodnie)

6.12 Sprint 11

Czas trwania Sprintu: 15.04.2026 - 29.04.2026 (2 tygodnie)

7 Testy

8 Zarządzanie projektem

8.1 Planowanie pracy zespołu i środki komunikacji

Na etapie planowania przyjęto iteracyjny model realizacji prac oparty na podejściu Agile, uzupełniony wybranymi praktykami SCRUM. Prace podzielono na krótkie iteracje o stałej długości: 2-tygodniowe sprinty (z kilkoma wyjątkami). Ustalono także cykliczne, cotygodniowe spotkania, w ramach których definiowano zakres zadań, monitorowano postęp, konsultowano nowe pomysły albo napotkane problemy oraz dokonywano przegląd rezultatów. Każde spotkanie wspierane było agendą spotkania oraz meeting minutes w celu efektywnego przebiegu obecnych jak i przyszłych spotkań.

8.2 Narzędzia wspomagające

Zespół projektowy komunikował się głównie przy użyciu komunikatora Discord na wydzielonym do tego serwerze. Do przechowywania oraz kontroli wersji kodu źródłowego systemu użyty został serwis GitHub z wydzielonym repozytorem dostępnym pod adresem url: <https://github.com/Labbyn/labbyn> . W celu przechowywania wszelkich wyprodukowanych podczas prac dokumentów, użyty został współdzielony Dysk Google. Do rejestrowania czasu pracy oraz zarządzaniu przydzielonymi zadaniami użyto serwisu Jira.

8.3 Harmonogram Prac

Ze względu na ograniczony czas na wyprodukowanie systemu, prace zostały podzielone na 5 etapów z umownymi datami rozpoczęcia i ukończenia etapu:

1. Koncept (1. Września 2025 - 1. Listopada 2025)
2. Rozpoczęcie prac (1. Listopada 2025 - 30. Listopada 2025)
3. MVP (1. Grudnia 2025 - 11. Stycznia 2026)
4. Wersja 1.0 (12. Stycznia 2026 - 1. Maja 2026)
5. Testy i poprawki (1. Maja 2026 - 20. Czerwca 2026)

W celu najbardziej efektywnego postępu prac, zespół zdecydował się na poniższy proces tworzenia zadań oraz przyrostów:

- Przyrosty będą opierały się na stworzonych i zaplanowanych zadaniach.
- Każde zadanie musi mieć przypisany Epik.
- Epiki będą tworzone na podstawie wymagań gdzie: 1 wymaganie = Epik.
- Epiki zostaną przypisane do konkretnego etapu w celu zaplanowania przyszłych prac.

9 Nakład Prac

9.1 Całościowy nakład prac

9.2 Indywidualny nakład prac

9.2.1 Adrian Kopczyński

9.2.2 Aleksander Stankowski

9.2.3 Patryk Kośmider

9.2.4 Ziemowit Orlikowski

10 Wnioski końcowe

11 Załączniki

Płyta CD z następującą zawartością:

- *pliki projektowe* – pliki składające się na całość projektu
 - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
 - źródło pracy inżynierskiej.
 - dokumenty projektowe powstałe podczas prac
- *Adrian Kopczyński_Aleksander Stankowski_Patryk Kośmider_Ziemowit Orlikowski_praca pisemna* – katalog zawierający plik PDF z pracą inżynierską.
- *Sprawozdanie z ankiety.docx* – plik zawierający sprawozdanie z przeprowadzonego badania ankietowego.

Spis rysunków

1.1	Rich Picture	6
3.1	Wykres kołowy uzyskanych odpowiedzi na pytanie 1.	11
3.2	Wykres kołowy uzyskanych odpowiedzi na pytanie 2.	12
3.3	Wykres kołowy uzyskanych odpowiedzi na pytanie 3.	12
3.4	Wykres uzyskanych odpowiedzi na pytanie 4.	13
3.5	Wykres uzyskanych odpowiedzi na pytanie 5.	14
3.6	Wykres uzyskanych odpowiedzi na pytanie 6.	15
3.7	Wykres uzyskanych odpowiedzi na pytanie 7.	16
3.8	Wykres uzyskanych odpowiedzi na pytanie 8.	17
3.9	Wykres uzyskanych odpowiedzi na pytanie 9.	17
3.10	Wykres uzyskanych odpowiedzi na pytanie 10.	18
3.11	Uzyskane odpowiedzi na pytanie 11.	18
3.12	Diagram głównych aktorów	20
3.13	Diagram przedstawiający przypadki użycia użytkownika	21
3.14	Diagram przedstawiający przypadki użycia administratora grupy	21
3.15	Diagram przedstawiający przypadki użycia administratora systemu	22
3.16	Diagram sekwencji przedstawiający logowanie użytkownika	23
3.17	Diagram sekwencji przedstawiający przetwarzanie operacji CRUD	24
5.1	Obsługa zapytań HTTP w module machines, odpowiadająca za stworzenie nowego obiektu	74
5.2	Logika biznesowa klasy RoomService, odpowiadająca za aktualizację wartości obiektu Room	75
5.3	Początek klasy PrometheusExecutor, zarządzającej komunikacją z Prometheus	76
5.4	Początek klasy AnsibleExecutor, zarządzającej komunikacją i egzekucją funkcjonalności Ansible	77
5.5	Repozytorium CategoryRepository, którego jedynym zadaniem jest komunikacja z bazą danych	78
5.6	Endpoint służący do tworzenia nowego przedmiotu w inwentarzu ze wstrzyknięciem mechanizmu kontekstu	79

5.7	Funkcja <code>create_async_engine</code> z pliku <code>database.py</code> , tworzy globalnie dostępny silnik bazodanowy	79
5.8	Klasa <code>RedisClientManager</code> , tworzy globalnie dostępna pojedynczą instancję do komunikacji z pamięcią podręczna.	80
5.9	Klasa modelująca tabele <code>User</code> z bazy danych do kodu	81
5.10	Schemat DTO, służący jako podstawowy schemat dla operacji związanych z użytkownikiem, powstały poprzez wykluczenie niepotrzebnych pól.	82
5.11	Obserwator rejestrujący zmiany dotyczące tworzenia nowych obiektów	83
5.12	Przykładowy plik <code>api.env</code> , konfiguruje parametry startowe aplikacji i zmienne globalne	84
5.13	Menedżer kontekstu inicjalizujący życie aplikacji	85
5.14	Kod inicjalizujący CORS oraz montowanie folderu dla plików statycznych	86
5.15	Kod podłączający poszczególne mniejsze routery, by stworzyć pełen interfejs API	87
5.16	Model bazodanowy reprezentujący obiekt <code>Machine</code> wraz z użyciem blokady optymistycznej i atrybutów obliczalnych	89
5.17	Przykładowy wygenerowany automatycznie w przypadku wprowadzenia zmian do modeli, skrypt migracyjny.	90
5.18	Konfiguracja autentykacji i strategii przechowywania tokenów	91
5.19	Klasa <code>UserManager</code> , poszerzona o możliwość logowania się za pomocą loginu	92
5.20	Klasa <code>AuthService</code> , serwis posiadający metody do resetowania hasła i obsługi pierwszego zalogowania	93
6.1	Zdjęcie przedstawiające panel monitorowania urządzeń systemu Prometheus	94
6.2	Zdjęcie przedstawiające integrację serwisów GitHub oraz Jira	95
6.3	Wstępny projekt modelu fizycznego bazy danych	96
6.4	POC systemu telemetry urządzeń	97
6.5	Wstępna wersja interfejsu użytkownika	97
6.6	Prototyp mapy przestrzennej laboratorium	98
6.7	Zdjęcie przedstawiające część mockupów stworzonych dla systemu Labbyn	98

Bibliografia

- [1] Damjan Maletic, Marta Grabowska i Matjaž Maletic. „Drivers and barriers of digital transformation in asset management”. W: *Management and Production Engineering Review* (2023), s. 118–126.